

Rashin Rahnamoun

400243092

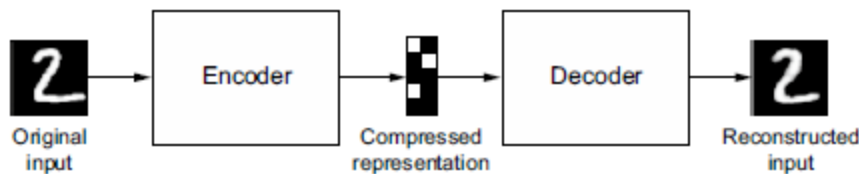
HW3_Vision

QA Report

Question 1:

A) Variational autoencoders (VAEs), independently introduced by Kingma and Welling in December 2013 and by Rezende, Mohamed, and Wierstra in January 2014, represent a type of generative model particularly well-suited for tasks such as image editing through the use of concept vectors. These models offer a modern interpretation of traditional autoencoders, integrating principles from both deep learning and Bayesian inference.

Autoencoders are neural networks designed to map input data to a low-dimensional latent space and subsequently reconstruct the input from this latent representation. In a typical image autoencoder, an encoder module compresses the image into a latent vector, while a decoder module reconstructs the image from this vector, producing an output with the same dimensions as the original image. The training process involves minimizing the difference between the reconstructed and input images, thereby enabling the autoencoder to learn to replicate the inputs. By imposing specific constraints on the latent representation—such as low dimensionality or sparsity—autoencoders can be tailored to discover meaningful latent features. In such cases, the encoder functions as a compression mechanism, reducing the input data into a more compact form.



B) Training a variational autoencoder involves optimizing two loss functions. The first is a *reconstruction loss*, which ensures that the decoded outputs closely resemble the original inputs. The second is a *regularization loss*, which encourages the learning of well-distributed latent representations while minimizing overfitting to the training data.

```
z_mean, z_log_variance = encoder(input_img)
```

```
z = z_mean + exp(z_log_variance) * epsilon
```

```
reconstructed_img = decoder(z)
```

```
model = Model(input_img, reconstructed_img)
```

In VAEs, training the model involves optimizing two distinct loss components: the reconstruction loss and the regularization loss. These losses work together to ensure that the model not only reconstructs input data accurately but also learns a well-structured latent space representation.

Reconstruction Loss

The reconstruction loss measures how well the decoder recreates the original input image from its latent representation. It compares the original input ('input_img') and the reconstructed output ('reconstructed_img') to ensure that the VAE learns to preserve the essential features of the input data.

This loss is typically computed using metrics such as Mean Squared Error (MSE) or Binary Cross-Entropy (BCE), depending on the nature of the data:

- **MSE** is commonly used for continuous data. It penalizes the squared differences between the input and the output pixel values.
- **BCE** is preferred for binary or normalized data (e.g., grayscale pixel intensities normalized between 0 and 1).

The reconstruction loss can be expressed as:

$$\mathcal{L}_{\text{reconstruction}} = \text{distance}(\text{input_img}, \text{reconstructed_img})$$

Regularization Loss (Kullback–Leibler Divergence)

The regularization loss encourages the latent space to follow a specific structure, typically a standard normal distribution ($N(0, I)$, where the mean is 0 and the variance is 1). This ensures the latent space is smooth, continuous, and structured, making it possible to sample meaningful points for generating new data.

The Kullback–Leibler (KL) divergence is used to quantify the difference between the encoder's output distribution and the target standard normal distribution. The encoder outputs two parameters, z_{mean} and $z_{\text{log_variance}}$, representing the mean and variance of the latent space. The KL divergence between the two distributions can be calculated as:

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2} \sum (1 + z_{\text{log_variance}} - z_{\text{mean}}^2 - \exp(z_{\text{log_variance}}))$$

This expression penalizes deviations of the latent distribution from the target normal distribution by:

1. Minimizing the squared mean, z_{mean}^2 , to center the latent space around 0.
2. Encouraging the variance $\exp(z_{\text{log_variance}})$, to approach 1, ensuring a consistent scale.
3. Adding a constant term to avoid numerical instability.

Combined Loss Function

The total loss function is a weighted sum of the reconstruction and regularization losses:

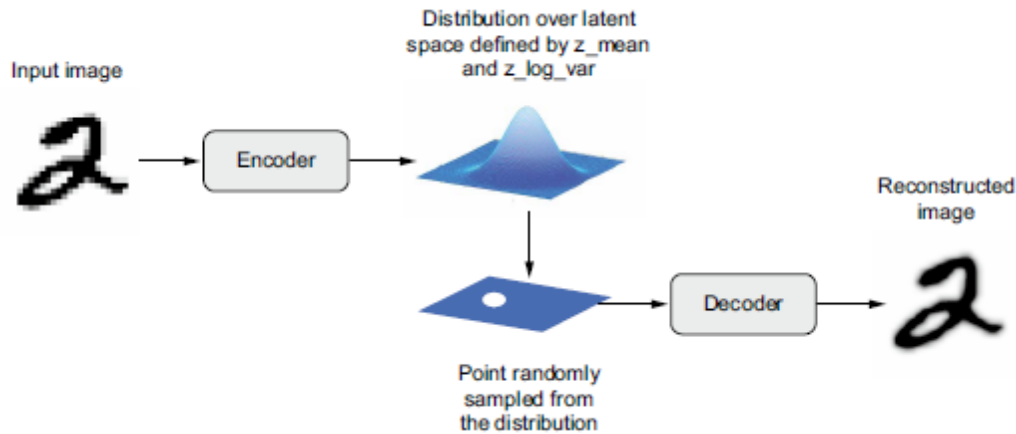
$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{reconstruction}} + \beta \cdot \mathcal{L}_{\text{KL}}$$

Here, β is a scaling factor (often set to 1 in standard VAEs) that balances the trade-off between accurately reconstructing inputs and enforcing structure in the latent space. Variants like β -VAEs adjust this factor to prioritize one aspect over the other.

By minimizing this combined loss, the VAE learns to reconstruct the input data while maintaining a well-structured and interpretable latent space.

C)

Unlike traditional autoencoders, VAEs encode input images into the parameters of a statistical distribution, specifically a mean and a variance, rather than a fixed latent code. This approach assumes that the input image is generated through a statistical process, and incorporates the inherent randomness of this process during both encoding and decoding. The VAE samples a point from the distribution defined by the mean and variance, and decodes this sample back to an approximation of the original input. This stochastic process enhances robustness and ensures that the latent space captures meaningful and continuous representations. As a result, any point sampled from the latent space corresponds to a valid output.



The functioning of a variational autoencoder (VAE) can be summarized as follows:

1. An encoder module turns the input sample, `input_img`, into two parameters in a latent space of representations, `z_mean` and `z_log_variance`.
2. You randomly sample a point `z` from the latent normal distribution that's assumed to generate the input image, via $z = z_mean + \exp(z_log_variance) * \epsilon$, where `epsilon` is a random tensor of small values.
3. A decoder module maps this point in the latent space back to the original input image.

The inclusion of randomness through *epsilon* ensures that any point near the latent representation of the *input_img* (`z_mean`) can be decoded into a representation similar to the *input_img*. This enforces continuity in the latent space, meaning that points located close to one another in the latent space correspond to highly similar outputs. When combined with the low dimensionality of the latent space, this continuity ensures that each direction within the space encodes a meaningful variation of the data. As a result, the latent space becomes well-structured and highly amenable to manipulation using concept vectors.

Question 2:

A) Generative adversarial networks (GANs), introduced by Goodfellow et al. in 2014, provide an alternative approach to variational autoencoders (VAEs) for learning latent spaces in images. GANs are particularly effective for generating realistic synthetic images by ensuring that the generated outputs are statistically indistinguishable from real images.

An intuitive way to grasp the concept of GANs is by imagining a scenario where a forger attempts to create counterfeit Picasso paintings. Initially, the forger produces poor imitations, which are mixed with genuine Picassos and presented to an art dealer for evaluation. The art dealer assesses the authenticity of each painting and provides feedback on what characteristics make a painting resemble a Picasso. Using this feedback, the forger refines their technique and produces better imitations. Over time, the forger becomes adept at replicating Picasso's style, while the art dealer improves at identifying fakes. Eventually, the forger's counterfeits become nearly indistinguishable from authentic works.

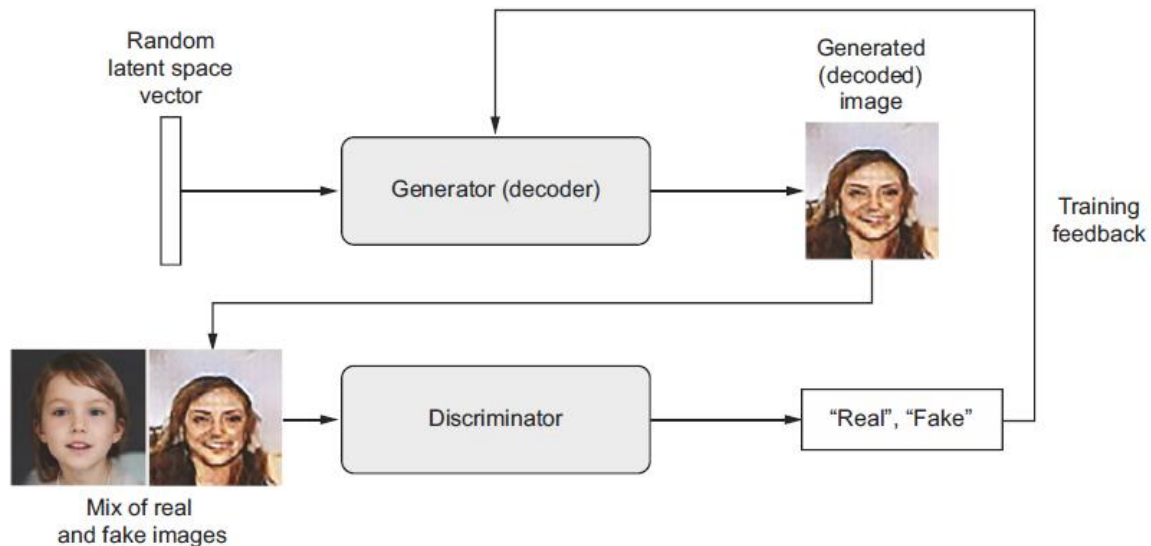
This dynamic illustrates the mechanism of a GAN, which consists of two interconnected components that compete and learn from each other:

- **Generator network:** This network starts with a random vector (a point in the latent space) and transforms it into a synthetic image.

- **Discriminator network (or adversary):** This network evaluates an image, whether real (from the training dataset) or synthetic (from the generator), and predicts its authenticity.

The generator is trained to deceive the discriminator by producing increasingly realistic images. As training progresses, the generator improves its ability to create images that the discriminator struggles to classify as fake. Simultaneously, the discriminator becomes more adept at distinguishing between real and synthetic images, continuously setting a higher standard for realism.

By the end of the training process, the generator can reliably convert any random point in the latent space into a plausible, realistic image. You can see the above explanation in the following diagram:



B) As explained above, Generative adversarial networks (GANs) rely on a unique, competitive framework to optimize two neural networks simultaneously: the generator and the discriminator. The loss functions of GANs are central to this competitive process, as they define the objectives of these two networks, enabling them to improve iteratively through their adversarial relationship.

The discriminator's role is to correctly distinguish real images (from the training data) from fake images (produced by the generator). To achieve this, the discriminator is trained to maximize its classification accuracy. The discriminator's loss function is expressed as:

Here:

$D(x)$: The discriminator's probability that a real image x is authentic.

$G(z)$: The generator's output when given a random input z .

$1 - D(G(z))$: The discriminator's probability that a fake image is classified as fake.

The discriminator minimizes this loss by correctly classifying real images as 1 and fake images as 0.

The generator's goal is to produce images that the discriminator classifies as real. The generator is trained to "fool" the discriminator by maximizing the likelihood that its synthetic images are classified as real. Its loss function is expressed as:

$D(G(z))$: The discriminator's probability that a generated image is real.

The competitive nature of GANs arises from their design as a zero-sum game:

The generator tries to minimize LGLG, aiming to fool the discriminator.

The discriminator tries to minimize LDLD, aiming to distinguish real from fake images.

This competition is crucial for the following reasons:

Mutual Improvement Through Feedback: The discriminator provides feedback to the generator about how realistic its outputs are. The generator uses this feedback to improve its ability to create images that better mimic real ones. Simultaneously, as the generator improves, the discriminator learns to become more discerning, creating a loop of mutual improvement.

Dynamic Equilibrium: The adversarial setup pushes both networks to a dynamic equilibrium. At this point, the generator produces images that are indistinguishable from real ones, and the discriminator cannot reliably differentiate between real and fake images (outputs a probability of 0.5 for both).

Encourages Realistic Outputs: The competitive loss ensures that the generator doesn't simply replicate training data but instead learns to approximate the underlying distribution of the real data. This leads to diversity in generated outputs.

C) Deep Convolutional GANs (DCGANs) : Imagine the potential of leveraging more advanced neural network architectures in GANs. Deep Convolutional GANs (DCGANs) take this idea further by replacing the simple, two-layer feed-forward networks typically used in basic GANs with convolutional neural networks (CNNs, also known as ConvNets) for both the generator and discriminator. This adaptation significantly enhances the model's ability to process image data, leveraging the spatial hierarchies and feature extraction capabilities inherent to CNNs.

There are main concepts in DCGAN as mentioned follows:

Use of Convolutional Neural Networks: In DCGAN, the generator and discriminator are designed as deep convolutional networks. CNNs are particularly well-suited for image-related tasks because they can capture spatial relationships between pixels, enabling the model to generate and discriminate more realistic and detailed images.

Generator: The generator uses transposed convolutions (also known as deconvolutions) to upsample a random noise vector into a high-resolution image. *Discriminator:* The discriminator uses standard convolutions to classify whether an input image is real or synthetic, learning to identify subtle features that distinguish real data from generated samples.

Batch Normalization: Introduced by Sergey Ioffe and Christian Szegedy in 2015, batch normalization is a technique used to normalize the inputs to each layer of the network within a mini-batch during training. This approach ensures that the data flowing through the network maintains a consistent distribution, which stabilizes and accelerates training. Batch normalization also mitigates issues like internal covariate shift, making it easier for the model to converge.

Just as input normalization improves the performance of neural networks, normalizing intermediate layer inputs enhances the training process of both the generator and discriminator in DCGANs. This normalization is applied to most layers in the network, except the output layer of the generator and the input layer of the discriminator.

Leaky ReLU Activation: DCGANs typically use the Leaky ReLU activation function in the discriminator instead of the standard ReLU. Leaky ReLU allows a small gradient to pass through even for negative inputs, addressing the problem of dead neurons and improving gradient flow during training.

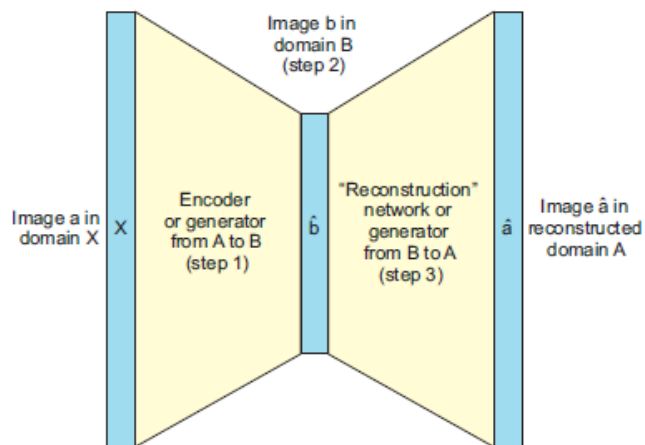
Tanh Activation in the Generator Output: The generator's output layer uses the Tanh activation function to scale the pixel values of the generated image between -1 and 1. This choice complements the normalization of real image data during preprocessing.

Elimination of Fully Connected Layers: DCGANs avoid using fully connected (dense) layers, relying solely on convolutions and transposed convolutions. This design choice reduces the number of parameters and ensures that the spatial structure of the images is preserved throughout the network.

CycleGAN :Building on the advanced concepts of Deep Convolutional GANs (DCGANs), we now explore CycleGANs, a powerful extension that combines ideas from conditional GANs (CGANs) and autoencoders. This innovative architecture allows for image-to-image translation between two distinct domains without requiring paired training data, making it particularly useful for applications like style transfer, domain adaptation, and image synthesis.

CycleGAN: A High-Level Overview

The CycleGAN architecture can be thought of as two CGANs interconnected to form a closed-loop structure. Alternatively, as the authors of CycleGAN describe, it resembles an autoencoder where the latent space corresponds to another meaningful image domain rather than an abstract representation.



In a traditional autoencoder, an input image (x) is compressed into a latent representation (z) and then reconstructed to produce an output (x^*). In the CycleGAN framework, the process involves two domains:

Domain A contains images like a , and Domain B contains images like b .

The model maps $a \rightarrow b \rightarrow a$ (denoted $F(G(a))$) and $b \rightarrow a \rightarrow b$ (denoted $G(F(b))$).

Here, the "latent space" between the domains is simply the other domain (A or B), making the mapping interpretable and ensuring high-dimensional, domain-specific transformations.

Main components of this model are as follows:

Adversarial Losses : Each mapping (from A to B and from B to A) includes a discriminator, which ensures that the translated images in each domain are indistinguishable from real images in that domain. These adversarial losses are derived from standard GAN objectives and guide the generators to produce high-quality outputs.

Cycle-Consistency Loss: The cycle-consistency loss ensures that translating an image into the opposite domain and back will reconstruct the original image as accurately as possible. For example:

$$\mathcal{L}_{\text{cycle}} = \|F(G(a)) - a\| + \|G(F(b)) - b\|$$

This loss helps preserve content and structure while transferring style between domains.

Interpretable Latent Space: Unlike traditional autoencoders, where the latent space is abstract, the latent space in CycleGANs corresponds to another domain, making the mappings intuitive and meaningful.

Two Autoencoders in Action: The architecture can be viewed as two autoencoders working in tandem, where the discriminators act as regulators to ensure realism in the translated images.

StyleGAN : Building on the principles of DCGANs and CycleGANs, StyleGAN represents a major leap in generative adversarial networks by introducing a novel architecture designed for high-quality, controllable image synthesis. Developed by NVIDIA in 2018, StyleGAN achieves state-of-the-art results in generating photorealistic images while allowing fine-grained control over the style and structure of the outputs.

Unlike standard GANs, which rely on a single latent vector to generate an image, StyleGAN introduces an intermediate latent space and separates the generation process into two components:

Style Control: Determines the overall appearance of the image, such as color, texture, or artistic style.

Content Structure: Encodes the spatial arrangement and geometry of the image elements, like the layout of a face or the position of objects.

This separation allows StyleGAN to disentangle style and content, enabling users to manipulate specific aspects of the generated images independently.

StyleGAN have different components:

Mapping Network: Instead of directly using the latent vector z sampled from a Gaussian distribution, StyleGAN maps z into an intermediate latent space w using a fully connected network. This mapping helps improve the disentanglement of features in the generated images, making the latent space more interpretable and controllable.

Adaptive Instance Normalization (AdaIN): StyleGAN introduces the AdaIN layer, which applies style information to each layer of the generator.

The intermediate latent vector w is used to compute scaling (γ) and shifting (β) parameters for instance normalization.

These parameters modulate the activations in the generator, allowing precise control over the style at different levels of the image. For instance, lower layers affect coarse features like pose and structure, while higher layers control finer details like texture and color.

Progressive Growing: StyleGAN employs a progressive growing technique for both training and generation, where images are initially synthesized at low resolution and gradually refined to higher resolutions. This strategy stabilizes training and improves image quality.

Noise Injection: To introduce stochastic variation and enhance realism, random noise is added at each layer of the generator. This allows StyleGAN to generate subtle variations in details, such as hair strands or facial

D) When deciding between VAEs and GANs for a specific image generation or manipulation task, it's essential to consider their strengths and limitations in relation to your goals. Here's a breakdown of conditions where one may outperform the other:

When VAEs Are Better

1. Structured Latent Space

- VAEs excel when you need a continuous, well-structured latent space for tasks like:
 - Image editing (e.g., face swapping, turning frowns into smiles).
 - Morphing between images in a smooth and interpretable way.
 - Animations that traverse the latent space, such as transitioning between two images or exploring variations systematically.
 - The continuity and structure of the latent space make VAEs more reliable for applications that require gradual, interpretable changes.
2. Ease of Training
- VAEs are generally easier to train compared to GANs. The training process involves optimizing a well-defined, single loss function that combines reconstruction loss and regularization (e.g., KL divergence). This stability makes VAEs a good choice for applications where robustness and simplicity are priorities.
3. Applications That Tolerate Slightly Less Realism
- While VAEs can produce convincing images, they may lack the hyper-realistic quality of GAN-generated outputs. However, if the task prioritizes latent space usability over photorealism (e.g., academic research or certain types of creative exploration), VAEs are often sufficient.

When GANs Are Better

1. Hyper-Realistic Image Generation
- GANs are the preferred choice when the primary goal is generating highly realistic images, as they can produce outputs that closely mimic real-world visuals. Examples include:
 - Creating photorealistic synthetic faces or objects.
 - Generating high-quality art or textures for video games and simulations.
 - Super-resolution and style transfer applications.
2. Static Single-Frame Outputs
- If you're not concerned with manipulating or traversing the latent space, GANs provide superior quality for generating single, standalone images.
3. Applications with Access to Expertise
- Training a GAN requires significant expertise due to:
 - The dynamic nature of its adversarial training (generator vs. discriminator).
 - The need for heuristic tuning and careful architecture choices.
 - For teams with experience in GANs or access to pre-trained models, GANs can be optimized to produce state-of-the-art results.

Question 3:

A) Diffusion models are based on the idea of generating complex data, such as images or audio, by learning to reverse a gradual process of adding noise to data. Inspired by thermodynamics, these models work through two interconnected stages: forward diffusion and reverse diffusion.

In the forward diffusion process, the model begins with a structured data sample and systematically adds small amounts of noise over multiple steps. With each step, the data becomes less recognizable, eventually transforming into what appears to be random noise. This process is controlled by a schedule that dictates the level of noise introduced at each step. By the end of the forward diffusion, the data structure is almost entirely destroyed, resembling a Gaussian distribution.

The reverse diffusion process begins with this noisy state and aims to reconstruct the original data by progressively removing the noise added during the forward process. To achieve this, a neural network is trained to predict the noise at each step based on the noisy input and subtract it to denoise the data progressively. This iterative denoising continues step by step until the final output closely resembles the original data sample. The reverse process requires the model to use information about each step of the process, often encoded as embeddings, to ensure accurate noise removal.

Overall, diffusion models create data by carefully learning how to reverse the destruction caused by noise. This approach allows for generating highly realistic data samples, as the iterative process provides precise control over the generation and reconstruction of the data.

B)

The forward process in diffusion models involves gradually corrupting structured data, such as an image, by adding noise at each step. This step-by-step transformation continues until the data becomes indistinguishable from pure random noise. The goal of this process is to create a path from structured data to a simple distribution, typically a Gaussian distribution, that the model can learn to reverse during the generation phase.

Gaussian noise is used because it has well-defined statistical properties and is mathematically tractable. Its continuous nature and simple probability density function make it convenient for theoretical analysis and optimization. Additionally, Gaussian distributions are universal approximators, meaning any distribution can be approximated as a mixture of Gaussians. This makes Gaussian noise an ideal choice for progressively corrupting data in a controlled and consistent manner.

The use of Gaussian noise allows for smooth and gradual corruption of data, enabling the forward process to be reversible and making it easier for the reverse process to accurately reconstruct the original data.

The noise addition in the forward process is mathematically defined using the following formulation:

This formulation allows the forward process to be analytically defined, facilitating the training of the reverse process and enabling efficient sampling during generation.

C) The reverse process is the core of diffusion models and focuses on reconstructing original data from noisy inputs by reversing the forward diffusion. It starts with data corrupted by Gaussian noise and progressively removes this noise over a sequence of steps, eventually generating a clean data sample that closely resembles the original input.

The reverse process can be seen as a denoising procedure, where the model learns to predict and subtract the noise added during the forward process. Mathematically, this is modeled using a reverse Markov chain:

The reverse process works because the neural network is trained to minimize the error between the predicted noise and the actual noise added during the forward process. This training ensures that the model accurately learns the statistical relationships between noisy data and the original data at each step, allowing for precise denoising. Over many iterations, the model reconstructs the original structure of the data, effectively converting random noise into a high-quality sample.

The iterative nature of the reverse process enables fine-grained control over the generation process, making diffusion models highly effective for tasks like realistic image generation and other complex data synthesis applications.

D)

The loss function in diffusion models is designed to train the neural network to predict the noise introduced during the forward process. This prediction is crucial for enabling the reverse process to reconstruct the original data accurately. The most common formulation of the loss function is based on the mean squared error (MSE) between the predicted noise and the actual noise added in the forward process.

The loss function is given as:

This loss ensures that the network learns to accurately predict the noise ϵ added during the forward process. The better the prediction of the noise, the more effectively the reverse process can denoise the data and reconstruct the original sample.

Why Learning Reverse Process Parameters is Important

Learning the reverse process parameters is critical because these parameters govern the quality and accuracy of the denoising steps. Key reasons include:

1. **Accurate Reconstruction:** The reverse process relies on the predicted noise at each timestep to denoise the data. If the parameters are not well-learned, the model cannot accurately subtract noise, resulting in poor reconstruction of the original data.
2. **Stability During Sampling:** The reverse process involves sampling from predicted Gaussian distributions at each step. Accurate parameters ensure that the generated samples follow the underlying data distribution, leading to stable and reliable outputs.
3. **Preservation of Data Structure:** The reverse process must not only denoise but also preserve the structure and features of the original data. Well-trained parameters allow the model to balance denoising and maintaining meaningful data representations.
4. **Robustness to Variability:** The forward process introduces varying amounts of noise at different timesteps. Learning the reverse process parameters ensures the model can adapt to these variations and apply the correct denoising strategies for each stage.
5. **Quality of Generated Samples:** The final quality of the generated data depends directly on how well the reverse process can reconstruct data from random noise. Properly learned parameters ensure high-quality, realistic outputs.

E)

The architecture of diffusion models revolves around the neural networks used to predict noise in the data during the reverse process. Several popular architectures have emerged, tailored to different data types and applications. Here are the main common architectures:

1. U-Net:

- Widely used in diffusion models, especially for image data.
- Combines convolutional layers for feature extraction with skip connections for preserving spatial details.
- Allows the model to learn both global and local structures of the data.
- U-Net is highly effective in denoising and reconstructing structured data like images.

2. ResNet (Residual Networks):

- Uses residual connections to allow gradients to flow easily through the network, aiding in deep model training.
- Suitable for tasks requiring deep networks, such as high-dimensional data processing.
- Often integrated into the encoder-decoder architecture for diffusion models.

3. Transformer-Based Architectures:

- Applied in text, audio, and image diffusion models.
- Leverages self-attention mechanisms to capture long-range dependencies in the data.
- Examples include models like **Denoising Diffusion Probabilistic Transformers (DDPT)** for high-resolution image generation.

4. Score-Based Models:

- These models use architectures like neural ODEs (Ordinary Differential Equations) to approximate data scores (gradients of log-density).
- Applied in score-matching formulations of diffusion models.

5. Conditional Architectures:

- Designed for conditional generation tasks, such as text-to-image synthesis.
- Inputs additional information (e.g., text embeddings) alongside noisy data for guided reconstruction.

Learning the Training Data Distribution

The network learns the data distribution by approximating the reverse process of the forward diffusion. This is achieved through:

1. Learning to Denoise:

- The network focuses on reconstructing the clean data x_0 from noisy inputs x_t . This implicitly helps it model the underlying data distribution.

2. Capturing Data Patterns:

- By minimizing the loss function over multiple iterations, the network learns the statistical patterns and structure of the training dataset.

3. Iterative Refinement:

- The training process teaches the model to perform incremental denoising steps, which cumulatively approximate the original data distribution.

Through this training approach, the diffusion model becomes capable of both denoising and generating new samples that resemble the training data.

F) The number of diffusion steps in a diffusion model plays a critical role in determining both the quality of the reconstructed data and the overall speed of the process. These steps control the gradual transformation of data through the forward process (adding noise) and the reverse process (removing noise to reconstruct the original data). The impact of these steps can be analyzed in terms of data quality and processing speed.

Effects on Reconstructed Data Quality

The quality of the reconstructed data is directly influenced by the number of diffusion steps. A higher number of steps allows for a finer-grained transformation of data between the original distribution and the noisy intermediate states. This gradual progression ensures that the model learns to reconstruct the data more accurately during the reverse process. Each step incrementally removes noise, and with more steps, the denoising becomes smoother and more precise. As a result, high-quality, detailed, and realistic outputs are achieved, especially when dealing with complex datasets.

However, there is a trade-off. If the number of steps is too low, the transformations between the noise-added states become abrupt, making it difficult for the model to accurately reverse the process. This can lead to artifacts or loss of detail in the reconstructed data, as the model lacks sufficient intermediate states to refine its predictions.

Effects on Process Speed

While increasing the number of diffusion steps improves output quality, it also significantly slows down the process. Each diffusion step involves a forward pass and a reverse pass through the neural network, so the computational cost grows linearly with the number of steps. This makes the inference time of diffusion models much longer compared to other generative models like GANs, which typically require only a single forward pass for generation.

G) Diffusion models, GANs, and VAEs are three prominent approaches for generative modeling, each with its own strengths and limitations, making them suitable for different tasks and priorities. Diffusion models stand out for their stability during training, as they avoid the adversarial setup of GANs, which often leads to challenges like mode collapse or vanishing gradients. These models generate high-quality and diverse samples by progressively denoising random noise into structured outputs, resulting in a well-defined latent space. This characteristic makes them highly flexible, capable of handling diverse data types such as images, audio, and text. However, diffusion models have notable drawbacks, such as slow sampling speeds due to the iterative nature of their generation process. Their training and inference are computationally expensive and require substantial resources, as well as careful design of noise schedules and architectures.

GANs, on the other hand, excel at generating sharp and photorealistic outputs, often producing the most visually convincing results among the three models. Their efficiency in sampling allows for quick generation of outputs after training, making them well-suited for tasks requiring real-time applications, such as super-resolution or style transfer. Despite these advantages, GANs face significant challenges in training due to their adversarial nature, which can lead to instability and extensive hyperparameter tuning. Furthermore, the latent space learned by GANs is less structured and continuous, limiting their utility in applications requiring interpretable or manipulable latent representations.

VAEs offer a different set of advantages, focusing on learning a continuous and interpretable latent space that facilitates tasks like interpolation, morphing, and concept vector manipulations. They are more stable and straightforward to train than GANs, benefiting from a probabilistic framework that incorporates uncertainty. However, this comes at the cost of output quality, as VAEs often produce blurry and less realistic results compared to GANs and diffusion models. Additionally, while their latent representations are structured, achieving higher-quality outputs often requires sacrificing some of this interpretability.

When choosing between these models, diffusion models are ideal for applications prioritizing high-quality and diverse outputs, especially when sampling speed is less critical. GANs are best suited for scenarios demanding photorealistic images and fast sampling, particularly in domains where sharpness and visual fidelity are essential. VAEs, with their structured latent spaces and stable training, are most effective for tasks involving interpolation, image editing, or creating smooth transitions. Ultimately, the choice of model depends on the specific requirements of the application and the trade-offs between quality, interpretability, and computational efficiency.

In summary :

Diffusion Models:

- Use when you need high-quality outputs and are not constrained by sampling speed.
- Suitable for diverse applications, including image generation, inpainting, and text-to-image synthesis, where diversity and structure are critical.

GANs:

- Ideal for applications requiring photorealistic images or tasks where sharpness and visual fidelity are paramount.
- Best when fast sampling is essential, such as real-time image generation.

VAEs:

- Best for tasks requiring structured and interpretable latent spaces, such as image editing, morphing, or creating smooth transitions between images.
- Use when stable and straightforward training is preferred, even at the cost of output sharpness.

Codes Report

Question 1:

Implementation Details of Variational Autoencoder for MNIST Dataset

Model Architecture

The implemented model is a **Variational Autoencoder (VAE)** designed to process and reconstruct MNIST images. The architecture consists of three main components: an encoder, a latent space, and a decoder. Each component is carefully structured to achieve effective representation learning and reconstruction.

```

# DEFINE VAE
class VAE(nn.Module):
    def __init__(self, latent_dim=2):
        super(VAE, self).__init__()
        self.encoder = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28 * 28, 400),
            nn.ReLU(),
            nn.Linear(400, 2 * latent_dim) # mu and log_var
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 400),
            nn.ReLU(),
            nn.Linear(400, 28 * 28),
            nn.Sigmoid()
        )
        self.latent_dim = latent_dim

    def reparameterize(self, mu, log_var):
        std = torch.exp(0.5 * log_var)
        eps = torch.randn_like(std)
        return mu + eps * std

    def forward(self, x):
        h = self.encoder(x)
        mu, log_var = h[:, :self.latent_dim], h[:, self.latent_dim:]
        z = self.reparameterize(mu, log_var)
        recon_x = self.decoder(z)
        return recon_x.view(-1, 1, 28, 28), mu, log_var

# Loss function
def loss_function(recon_x, x, mu, log_var):
    bce_loss = nn.functional.binary_cross_entropy(recon_x, x, reduction='sum')
    kld = -0.5 * torch.sum(1 + log_var - mu.pow(2) - log_var.exp())
    return bce_loss + kld, bce_loss, kld

```

1. Encoder

- **Input:** The model accepts MNIST images of shape (28, 28), which are flattened into a 1D vector of size 784.
- **Structure:**
 - A fully connected layer with 400 neurons, followed by a ReLU activation function.
 - A second fully connected layer that outputs $2 \times \text{latent_dim}$ values:
 - Half of these values correspond to the mean vector (μ).
 - The other half corresponds to the log variance vector (σ^2).

- **Purpose:** The encoder compresses the input image into a latent representation consisting of a mean and variance for each dimension in the latent space.

2. Latent Space

- **Reparameterization Trick:** The encoder output is used to sample latent vectors z by applying the following formula:

$$z = \mu + \epsilon \cdot \sigma, \epsilon \sim \mathcal{N}(0,1)$$

This trick allows the model to backpropagate gradients through the stochastic sampling process.

- **Purpose:** Ensures that the latent space captures meaningful distributions of the input data.

3. Decoder

- **Input:** The sampled latent vector z .
- **Structure:**
 - A fully connected layer with 400 neurons, followed by a ReLU activation function.
 - A second fully connected layer with 784 outputs, reshaped into (28, 28).
 - A Sigmoid activation is applied to constrain the output values between 0 and 1.
- **Purpose:** Reconstructs the input image from the latent representation.
-

Evaluation Function

The evaluation of the VAE involves a combination of a composite loss function and qualitative visualizations to assess reconstruction quality and latent space exploration.

1. Loss Function

The VAE loss is a combination of:

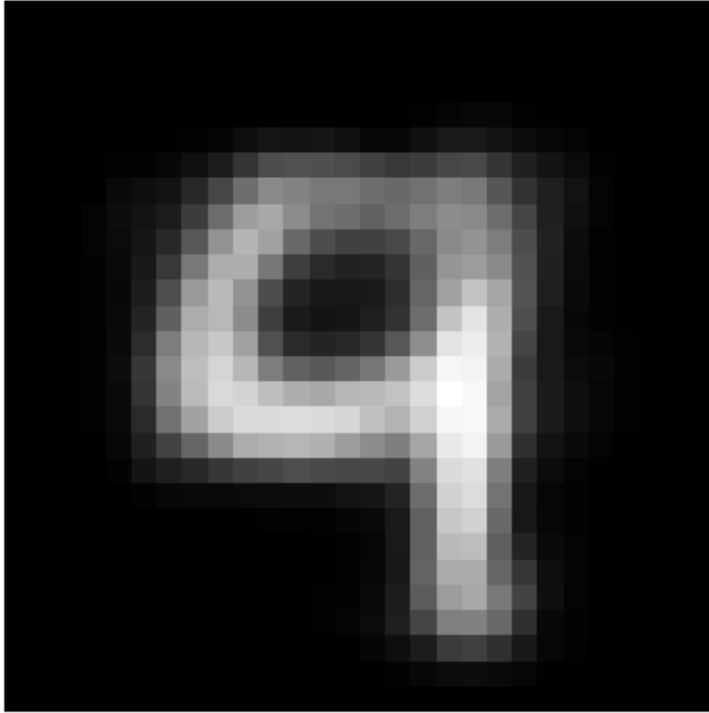
1. **Reconstruction Loss:**
 - Measures the difference between the input image and its reconstruction using binary cross-entropy (BCE): $BCE Loss = -\sum [x \cdot \log(x) + (1 - x) \cdot \log(1 - x)]$
 - Encourages the decoder to generate images close to the original input.
2. **KL Divergence:**
 - Encourages the latent space distribution to approximate a standard normal distribution: $KL Divergence = -0.5$
 - Ensures regularization of the latent space to prevent overfitting.
3. **Total Loss:**
 - Combines the reconstruction loss and KL divergence: $Total Loss = BCE Loss + KL Divergence$

2. Visualization

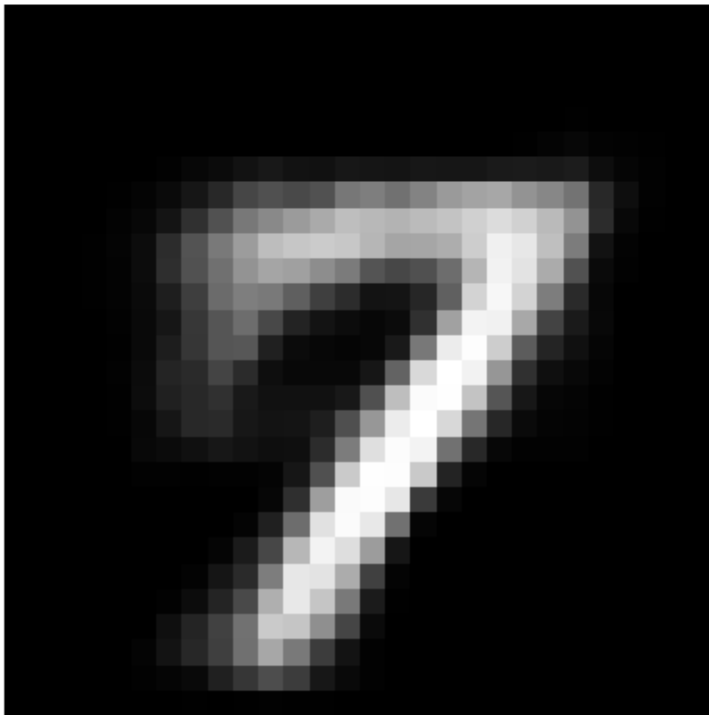
- **Latent Space Exploration:**

- The latent vectors are visualized using t-SNE to reduce the high-dimensional latent space into a 2D representation. This enables us to observe clustering patterns corresponding to different digits.
- **Reconstruction Quality:**
 - The model's ability to reconstruct images is assessed by comparing original images with their reconstructed counterparts side-by-side.
 - This provides insights into the fidelity and generalization capabilities of the model.

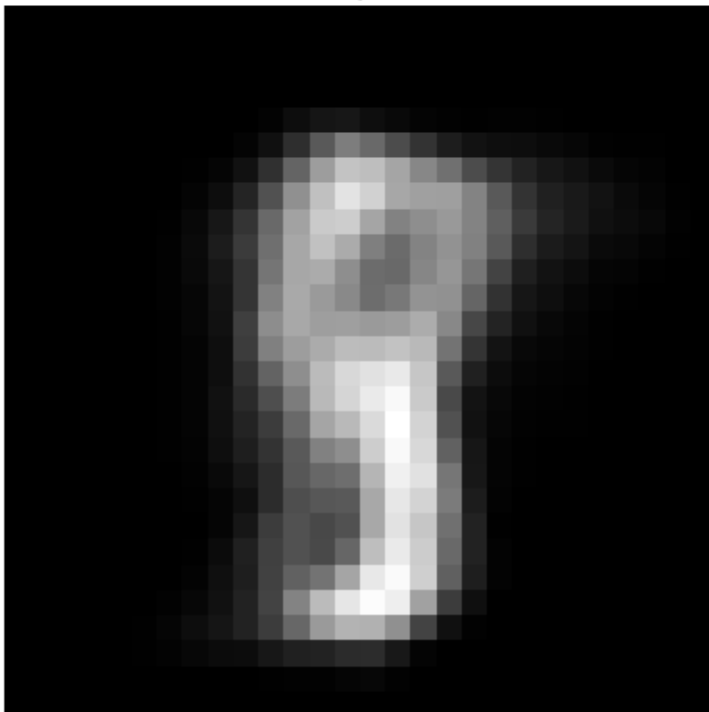
Sample 1



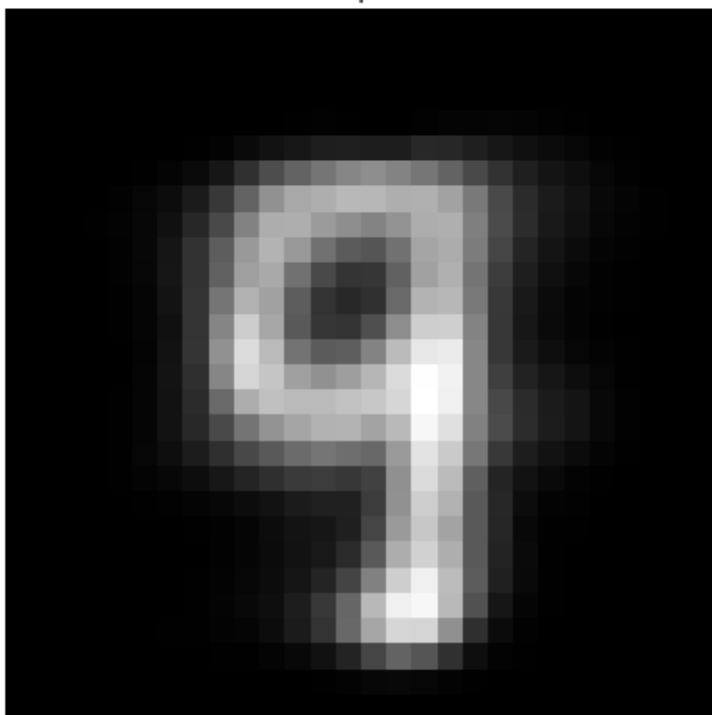
Sample 2



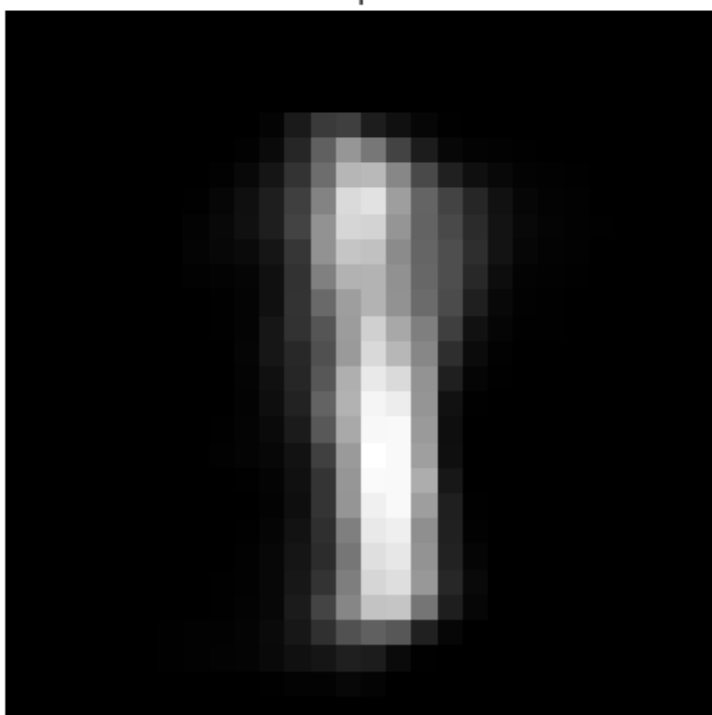
Sample 3



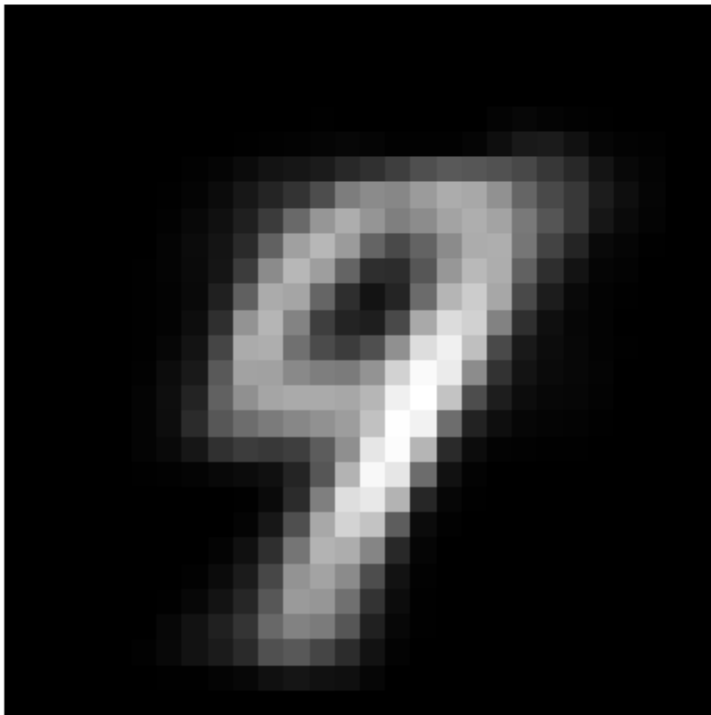
Sample 4



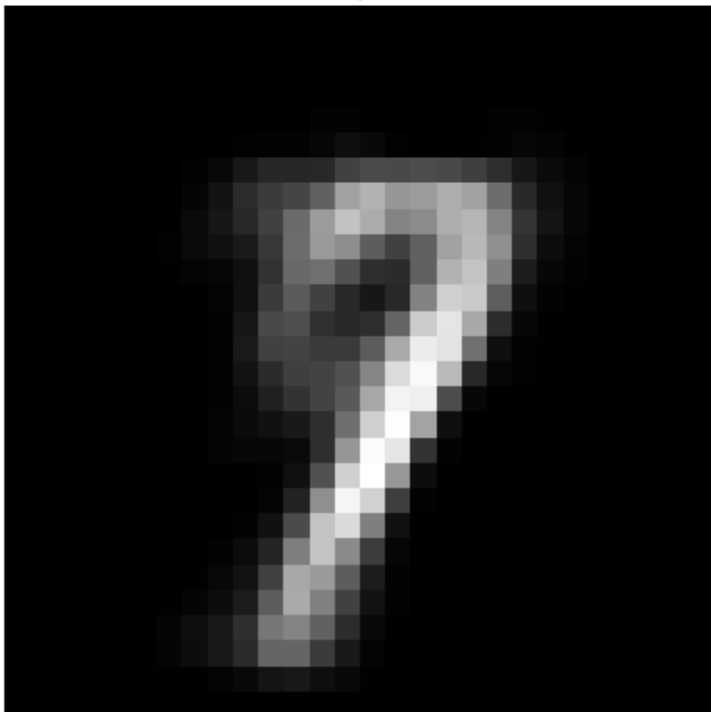
Sample 5



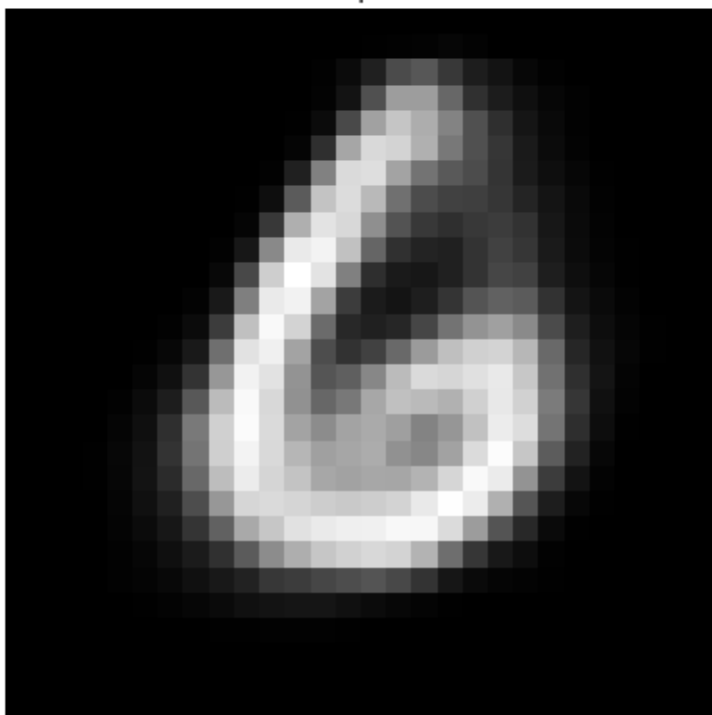
Sample 6



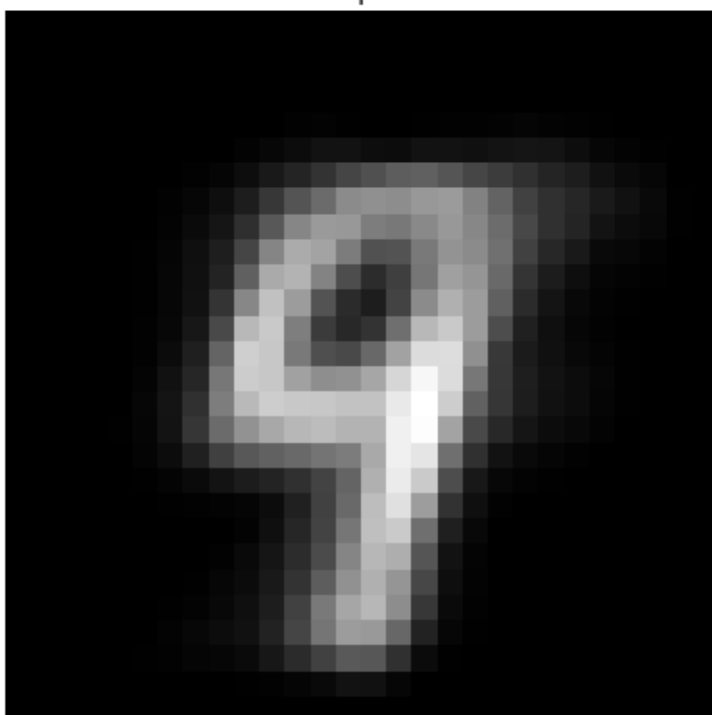
Sample 7

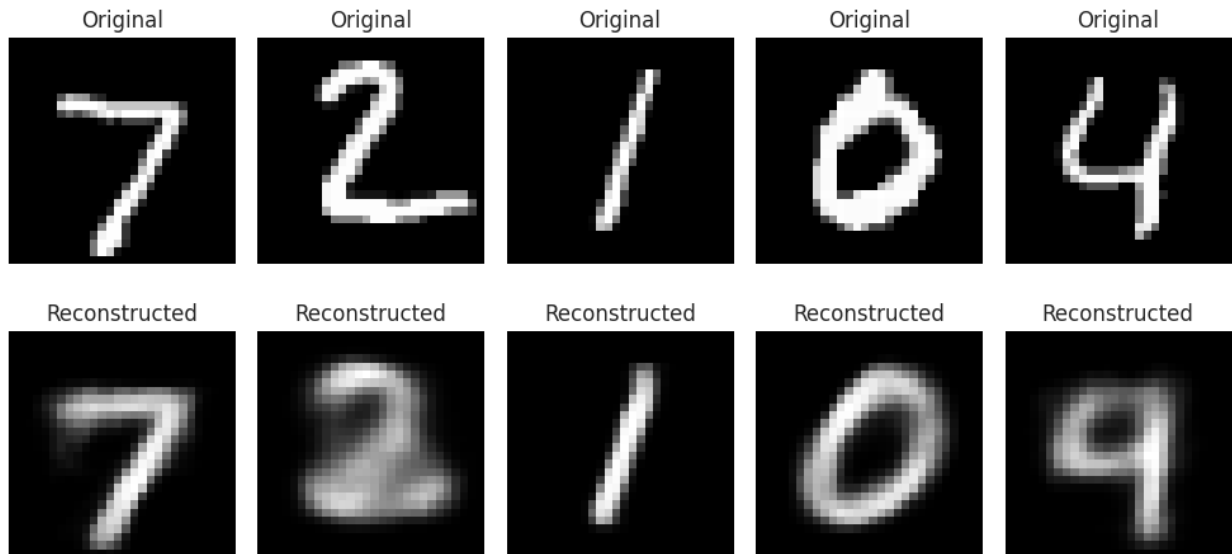


Sample 8



Sample 9





Question 1(Points and first part results):

Latent Dim 2 - Epoch 1, Loss: 191.1072

Latent Dim 2 - Epoch 2, Loss: 169.2805

Latent Dim 2 - Epoch 3, Loss: 164.0144

Latent Dim 2 - Epoch 4, Loss: 161.1624

Latent Dim 2 - Epoch 5, Loss: 159.2078

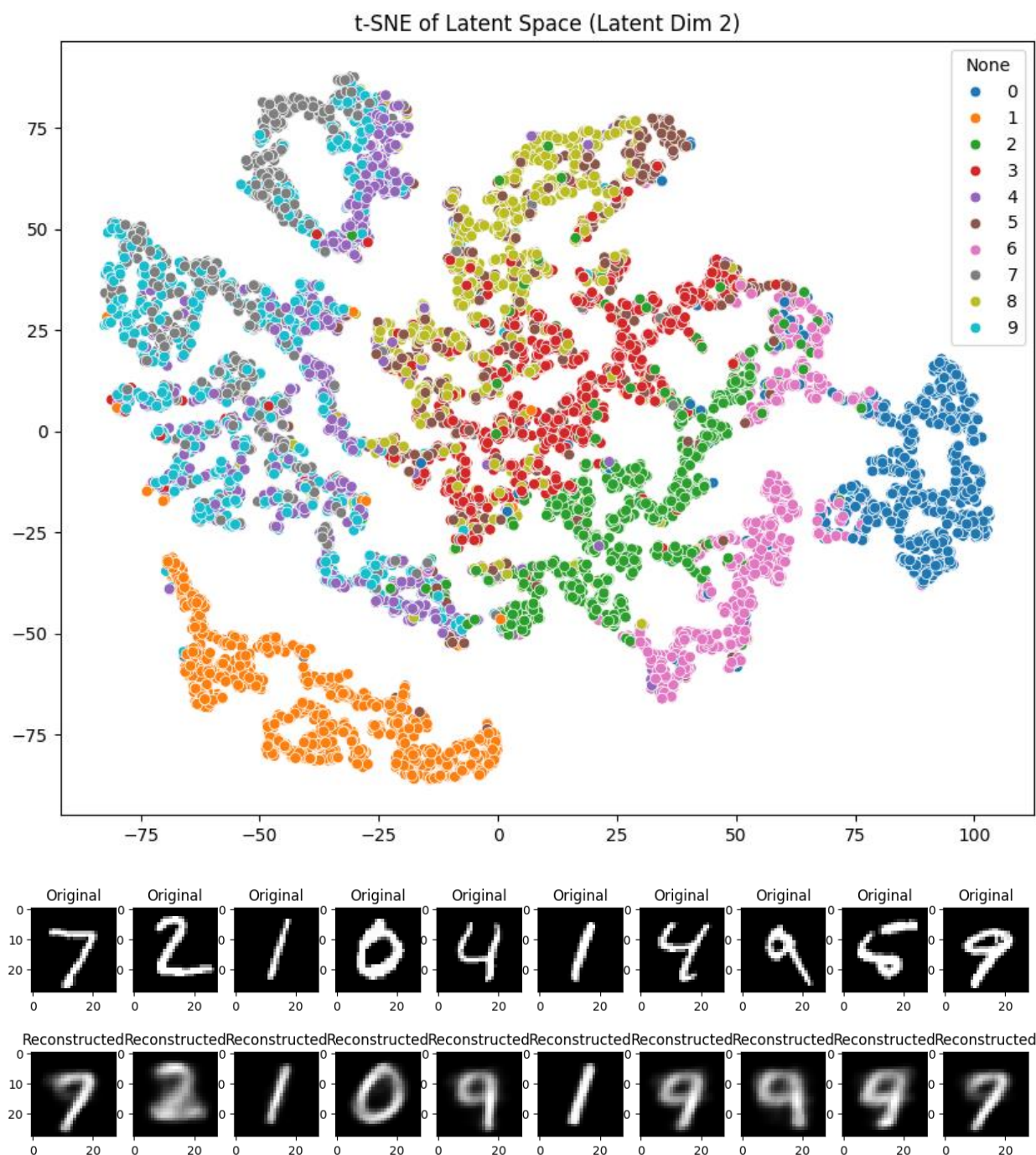
Latent Dim 2 - Epoch 6, Loss: 157.7852

Latent Dim 2 - Epoch 7, Loss: 156.6166

Latent Dim 2 - Epoch 8, Loss: 155.6531

Latent Dim 2 - Epoch 9, Loss: 154.9370

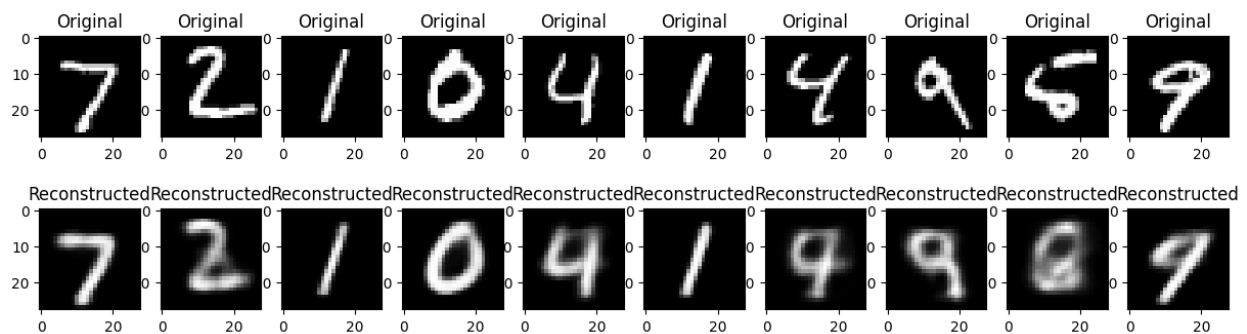
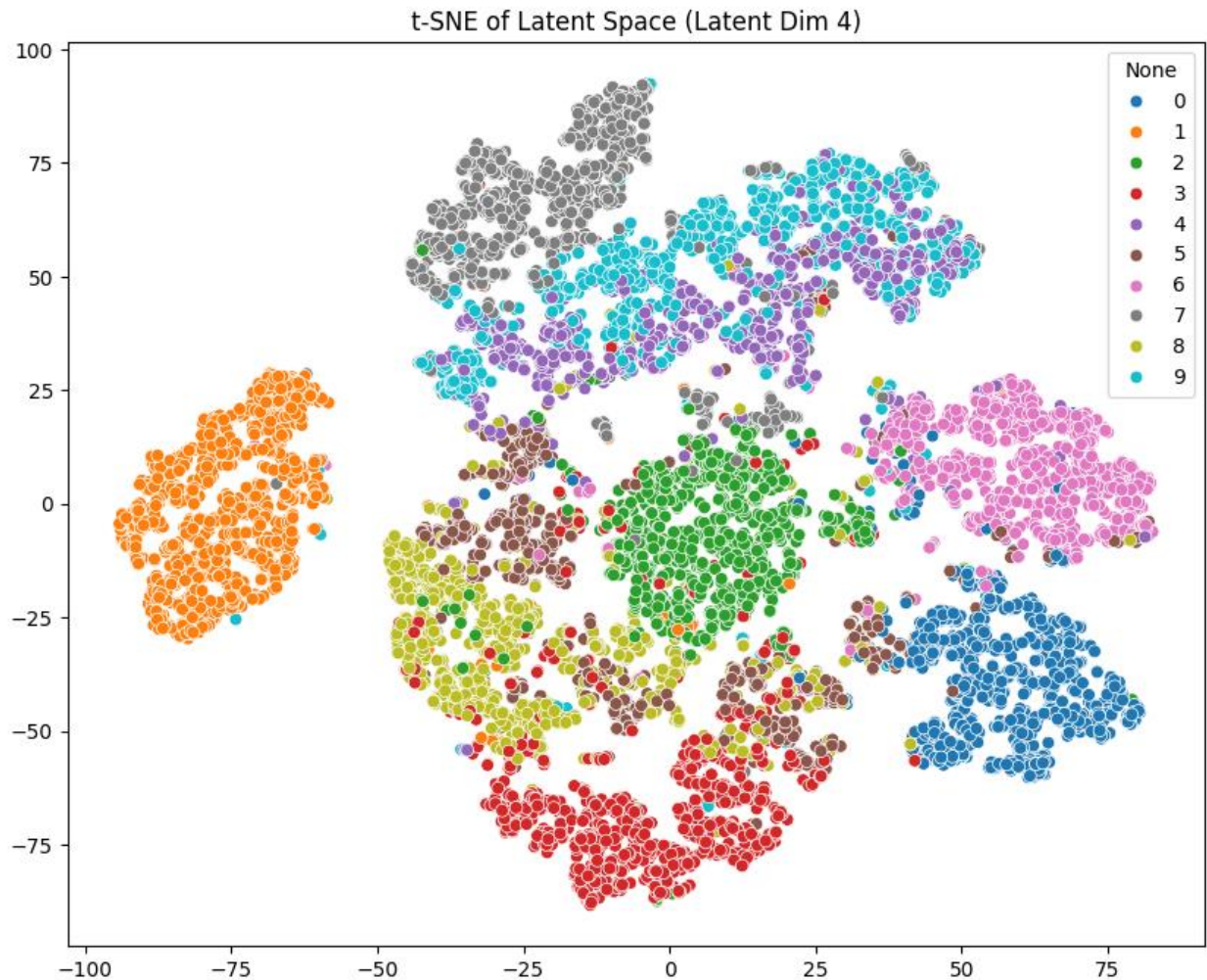
Latent Dim 2 - Epoch 10, Loss: 154.2356



Latent Dim 4 - Epoch 1, Loss: 171.8606

Latent Dim 4 - Epoch 2, Loss: 145.9617

Latent Dim 4 - Epoch 3, Loss: 141.6923
Latent Dim 4 - Epoch 4, Loss: 139.1017
Latent Dim 4 - Epoch 5, Loss: 137.2747
Latent Dim 4 - Epoch 6, Loss: 135.8831
Latent Dim 4 - Epoch 7, Loss: 134.7913
Latent Dim 4 - Epoch 8, Loss: 133.9006
Latent Dim 4 - Epoch 9, Loss: 133.1158
Latent Dim 4 - Epoch 10, Loss: 132.4827



Latent Dim 16 - Epoch 1, Loss: 164.1285

Latent Dim 16 - Epoch 2, Loss: 121.3999

Latent Dim 16 - Epoch 3, Loss: 115.1912

Latent Dim 16 - Epoch 4, Loss: 112.3251

Latent Dim 16 - Epoch 5, Loss: 110.6736

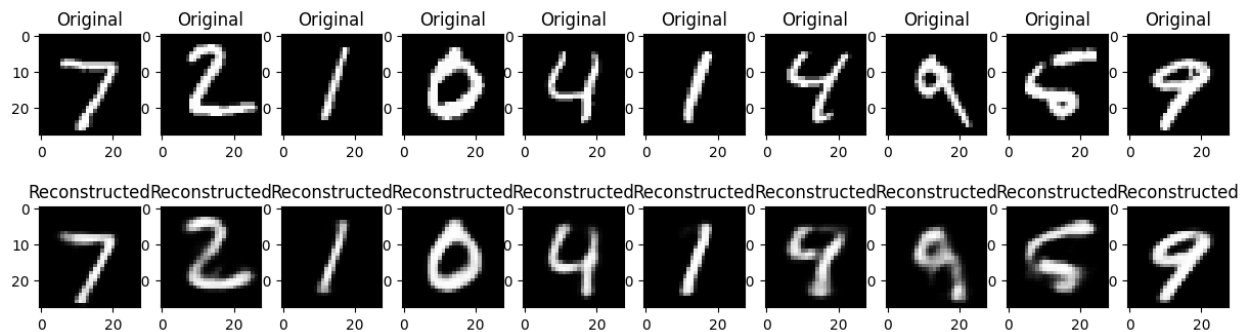
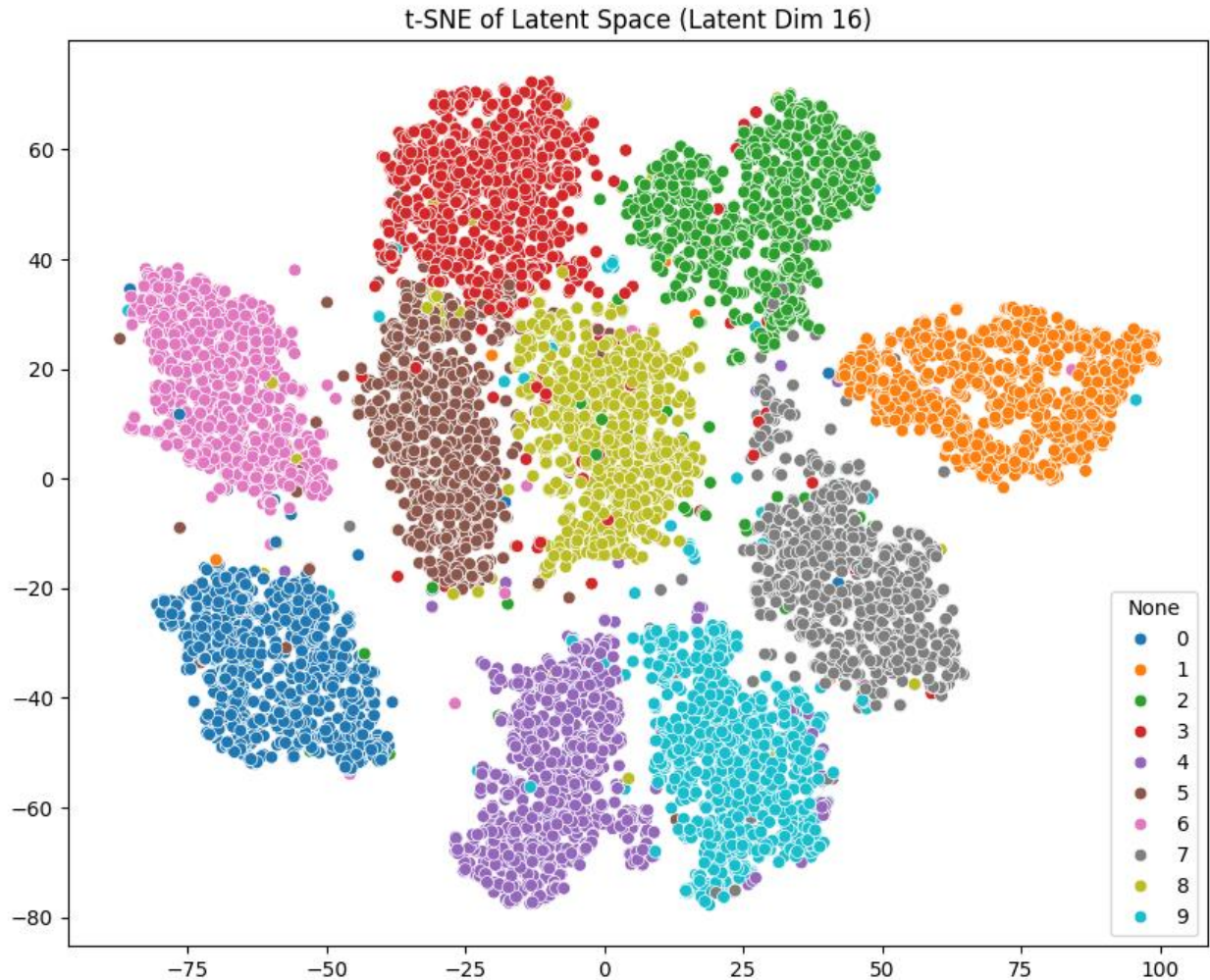
Latent Dim 16 - Epoch 6, Loss: 109.4848

Latent Dim 16 - Epoch 7, Loss: 108.6313

Latent Dim 16 - Epoch 8, Loss: 107.9676

Latent Dim 16 - Epoch 9, Loss: 107.4659

Latent Dim 16 - Epoch 10, Loss: 106.9837



Latent Space and the Effect of Latent Dimension Size on Constructed Images (Dimensions: 2, 4, 16)

The latent space in a Variational Autoencoder (VAE) serves as a compressed representation of the input data. It captures the essential features required to reconstruct the original images and is key to generating new samples. The size of the latent space (or latent dimension) directly affects the quality of reconstructed images and the VAE's ability to generate diverse and realistic samples.

Here, we analyze how latent dimension sizes of 2, 4, and 16 influence the performance of the VAE on the MNIST dataset.

1. Latent Dimension Size = 2

- **Structure of the Latent Space:**
 - With only two dimensions, the latent space is highly compressed. Each input digit is mapped to a point or small cluster in a 2D space.
 - This setup is ideal for visualization, as the relationships between clusters (e.g., different digit classes) can be easily plotted and observed.
- **Quality of Reconstructed Images:**
 - The reconstructions tend to be blurry or lack fine details because the limited dimensionality restricts the VAE's capacity to encode all necessary features of the input images.
 - Subtle variations in digits, such as stroke thickness or rotation, may not be well captured.
- **Generated Images:**
 - Generated images often exhibit basic shapes and outlines of digits but lack diversity and intricate features.
 - Overlapping clusters in the latent space can lead to mixed or ambiguous digit generation.

2. Latent Dimension Size = 4

- **Structure of the Latent Space:**
 - The increase in dimensionality allows the VAE to capture more nuanced features of the input data. Each digit class can now occupy a more distinct region of the latent space.
 - The structure remains interpretable to some extent, though visualizing relationships becomes harder as dimensionality grows.
- **Quality of Reconstructed Images:**
 - Reconstructions improve significantly compared to a 2-dimensional latent space. Key features like edges, stroke styles, and digit clarity are better represented.
 - The VAE can more accurately capture variations within each digit class.
- **Generated Images:**
 - Generated digits become more diverse and realistic. While still not perfect, the additional latent dimensions provide room for encoding meaningful variations in digit style and orientation.
 - Ambiguities in generation reduce as clusters in the latent space separate further.

3. Latent Dimension Size = 16

- **Structure of the Latent Space:**

- A 16-dimensional latent space provides ample capacity to represent the complexities of the MNIST dataset. Each digit class is mapped to a highly distinct region, and intra-class variations are well captured.
- However, the interpretability of the latent space diminishes, as it is no longer possible to easily visualize relationships.
- **Quality of Reconstructed Images:**
 - Reconstructions become highly accurate, closely resembling the original input images. Fine details, such as subtle curves, stroke thickness, and digit-specific features, are well-preserved.
 - Overfitting is avoided due to the regularization effect of the VAE's Kullback-Leibler divergence loss.
- **Generated Images:**
 - Generated digits exhibit excellent diversity and high fidelity, as the VAE can leverage the richer latent space to explore a wider range of variations.
 - However, if the dataset's complexity is low (as in MNIST), some dimensions of the latent space may remain underutilized, potentially leading to redundancy.

Conclusion

The implementation of the VAE for the MNIST dataset demonstrates the power of probabilistic modeling in unsupervised learning. The encoder and decoder effectively transform and reconstruct input images, while the latent space provides meaningful representations of the data. The reparameterization trick ensures that gradients can propagate through stochastic processes, enabling efficient training. Evaluation results, including reconstruction accuracy and latent space visualization, highlight the model's capability to learn and generalize effectively. This implementation serves as a strong foundation for extending VAEs to more complex datasets and tasks.

Question 2:

The code architecture is designed for image denoising by combining the strengths of a U-Net-based generator and a GAN-based discriminator, providing both pixel-level and perceptual loss functions. Here is a more technical breakdown of the architecture:

```
class Conv_block(nn.Module):
    def __init__(self, in_channels, out_channels, size=3, padding=1, dropout=0.2,
stride=1):
        super().__init__()
        self.convs = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, size, padding=padding),
            nn.LeakyReLU(0.1, inplace=True),
```

```

        nn.Dropout(dropout),
        nn.BatchNorm2d(out_channels),

        nn.Conv2d(out_channels, out_channels, size, padding=padding,
stride=stride),
        nn.LeakyReLU(0.1, inplace=True),
        nn.Dropout(dropout),
        nn.BatchNorm2d(out_channels),
    )

    def forward(self, x):
        return self.convs(x)

class DeConv_block(nn.Module):
    def __init__(self, in_channels, out_channels, size=3, padding=1,
dropout=0.2):
        super().__init__()
        self.deConv = nn.ConvTranspose2d(in_channels, out_channels, size,
padding=1, stride=2, output_padding=1)
        self.convs = Conv_block(out_channels * 2, out_channels, size, padding,
dropout)

    def forward(self, x, residual):
        x = self.deConv(x)
        x = torch.cat([x, residual], 1)
        return self.convs(x)

class Unet(nn.Module):
    def __init__(self, d=64, out_channels=3, dropout=0.2):
        super().__init__()

        # Unet encoder
        self.conv_blocks = nn.ModuleList([
            Conv_block(3, d, dropout=dropout),
            Conv_block(d, 2 * d, stride=2, dropout=dropout),
            Conv_block(2 * d, 4 * d, stride=2, dropout=dropout),
            Conv_block(4 * d, 8 * d, stride=2, dropout=dropout),
            Conv_block(8 * d, 16 * d, stride=2, dropout=dropout)
        ])

        # Unet decoder
        self.deConv_blocks = nn.ModuleList([
            DeConv_block(16 * d, 8 * d, dropout=dropout),

```

```

        DeConv_block(8 * d, 4 * d, dropout=dropout),
        DeConv_block(4 * d, 2 * d, dropout=dropout),
        DeConv_block(2 * d, d, dropout=dropout),
    ])

    self.output_conv = nn.Conv2d(d, out_channels, 3, padding=1)

    def forward(self, x):
        x1 = self.conv_blocks[0](x)
        x2 = self.conv_blocks[1](x1)
        x3 = self.conv_blocks[2](x2)
        x4 = self.conv_blocks[3](x3)
        x5 = self.conv_blocks[4](x4)

        x = self.deConv_blocks[0](x5, x4)
        x = self.deConv_blocks[1](x, x3)
        x = self.deConv_blocks[2](x, x2)
        x = self.deConv_blocks[3](x, x1)

        x = self.output_conv(x)
        return x

# Define Discriminator
class Discriminator(nn.Module):
    def __init__(self, label_channels=3, input_channels=3, d=64):
        super().__init__()

        self.convs = nn.Sequential(
            nn.Conv2d(label_channels + input_channels, d, 3, 2, 1),
            nn.LeakyReLU(0.1, inplace=True),
            nn.BatchNorm2d(d),

            nn.Conv2d(d, d*2, 3, 2, 1),
            nn.LeakyReLU(0.1, inplace=True),
            nn.BatchNorm2d(d*2),

            nn.Conv2d(d*2, d*4, 3, 2, 1),
            nn.LeakyReLU(0.1, inplace=True),
            nn.BatchNorm2d(d*4),

            nn.Conv2d(d*4, d*8, 3, 2, 1),
            nn.LeakyReLU(0.1, inplace=True),
            nn.BatchNorm2d(d*8),

```

```

        nn.Conv2d(d*8, 1, 3, 2, 1),
    )

    def forward(self, input, label):
        x = torch.cat([input, label], 1)
        x = self.convs(x)
        return x

# Dataset class for noisy and clean images
class ImageDenoisingDataset(Dataset):
    def __init__(self, noisy_dir, clean_dir, noise_levels, transform=None):
        self.noisy_dir = noisy_dir
        self.clean_dir = clean_dir
        self.noisy_images = sorted(os.listdir(noisy_dir))
        self.clean_images = sorted(os.listdir(clean_dir))

        # Ensure the number of noisy and clean images are the same
        min_images = min(len(self.noisy_images), len(self.clean_images))
        self.noisy_images = self.noisy_images[:min_images]
        self.clean_images = self.clean_images[:min_images]

        self.noise_levels = noise_levels # List of noise levels (e.g., [0.05,
0.1, 0.15])
        self.transform = transform

    def __len__(self):
        return len(self.noisy_images)

    def __getitem__(self, idx):
        noisy_img_path = os.path.join(self.noisy_dir, self.noisy_images[idx])
        clean_img_path = os.path.join(self.clean_dir, self.clean_images[idx])

        noisy_img = Image.open(noisy_img_path).convert('RGB')
        clean_img = Image.open(clean_img_path).convert('RGB')

        # Add random noise based on selected noise level
        noise_level = random.choice(self.noise_levels)
        noisy_img = self.add_noise(noisy_img, noise_level)

        if self.transform:
            noisy_img = self.transform(noisy_img)
            clean_img = self.transform(clean_img)

        return noisy_img, clean_img

```

```

def add_noise(self, img, noise_level):
    """
    Add random Gaussian noise to an image.
    noise_level: the standard deviation of the Gaussian noise
    """
    img = np.array(img) / 255.0 # Normalize to [0, 1]
    noise = np.random.normal(0, noise_level, img.shape) # Add Gaussian noise
    noisy_img = img + noise
    noisy_img = np.clip(noisy_img, 0, 1) # Ensure the values stay between
[0, 1]
    return Image.fromarray((noisy_img * 255).astype(np.uint8)) # Convert
back to uint8

```

1. Overview of GAN

GANs consist of two networks that play an adversarial game:

Generator (G): Learns to map noisy images to clean images.

Discriminator (D): Attempts to differentiate between real (clean) images and fake (generated) images.

The training process is a minimax game where:

The generator aims to minimize the discriminator's ability to classify its outputs as fake.

The discriminator tries to maximize its accuracy in distinguishing real from generated images.

2. Generator: U-Net

The generator in this code is implemented using a U-Net architecture, which is well-suited for tasks requiring high-quality image reconstruction. The key features are:

a. Encoder (Downsampling Path):

The encoder consists of five sequential convolutional blocks, implemented using the `Conv_block` class.

Each block:

Extracts high-level features through two convolutional layers.

Employs LeakyReLU activation, dropout, and batch normalization for efficient training.

Uses strides for downsampling, reducing the spatial dimensions.

b. Decoder (Upsampling Path):

The decoder mirrors the encoder and reconstructs the image to its original resolution.

Uses transposed convolutions (DeConv_block) for upsampling.

Employs skip connections to incorporate low-level spatial details from the encoder, ensuring fine-grained reconstruction.

c. Output Layer:

A final convolutional layer reduces the number of channels to match the target output (e.g., 3 for RGB images).

The generator learns to map noisy input images to clean target images, minimizing a loss function that combines adversarial loss (from the discriminator) and a reconstruction loss (e.g., Mean Squared Error or PSNR).

3. Discriminator

The discriminator is implemented as a Convolutional Neural Network (CNN) to classify image pairs (noisy input + clean image) as real or fake. The architecture comprises:

Sequential convolutional layers with increasing feature depth (d, 2d, 4d, etc.).

LeakyReLU activation for stable gradient flow.

Batch normalization for improved convergence.

The discriminator takes as input:

The noisy input image concatenated with:

Either the clean target image (real pair) or

The generator's output (fake pair).

It outputs a single-channel feature map, representing the classification probabilities for real vs. fake at various spatial locations.

5. Noise-Level Management:

The architecture is designed to handle multiple levels of noise by training separate models for each noise level. This is achieved by training and validating distinct generator and discriminator networks for each noise intensity, ensuring the model can generalize across varying levels of noise in the input images.

6. Checkpointing:

The code saves the best-performing model at each noise level based on the validation loss. This prevents overfitting to specific noise levels and ensures the model maintains optimal performance across various noise intensities.

7. Visualization and Evaluation:

Visualization: After training, the results are visualized by displaying a batch of images, showing noisy images, clean images, and their corresponding denoised images. This helps in qualitatively assessing the model's performance in removing noise while preserving fine details.

Quantitative Evaluation (PSNR): For each image in the batch, PSNR is calculated to evaluate the effectiveness of the denoising process. The average PSNR value for the batch serves as a numerical indicator of the model's performance in terms of image quality.

The charts below provide the following insights:

MSE Progression Across Epochs:

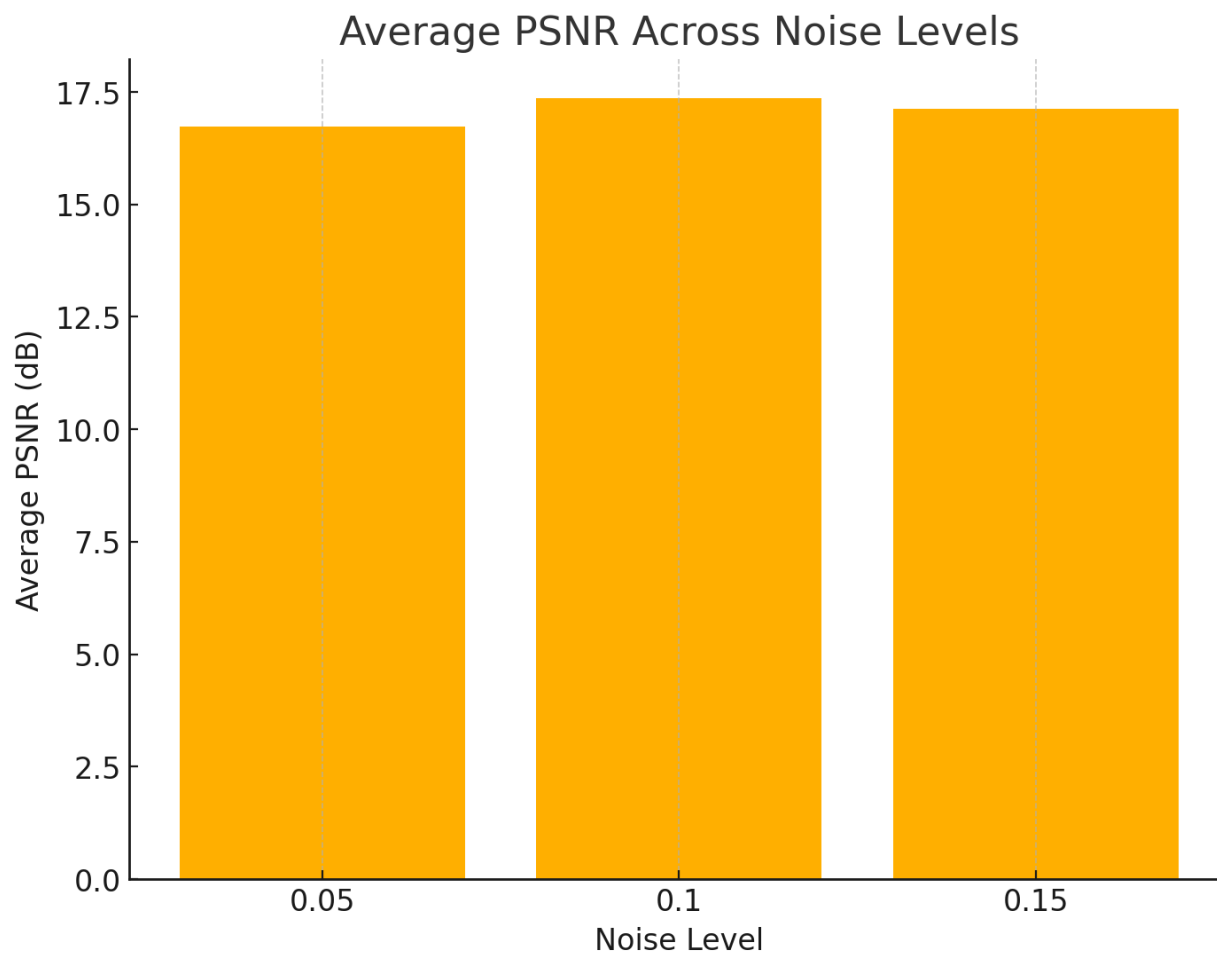
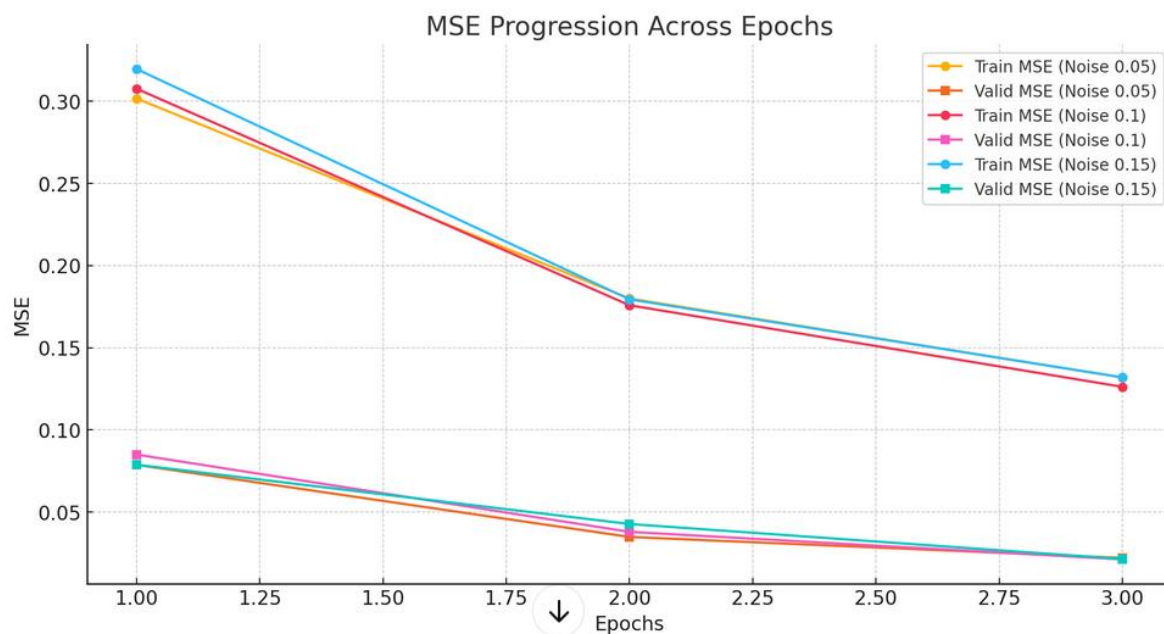
Both training and validation MSE decrease consistently over epochs for all noise levels.

The validation MSE closely follows the training MSE, indicating effective model generalization.

Average PSNR Across Noise Levels:

The highest average PSNR is achieved for a noise level of 0.1.

For noise levels 0.05 and 0.15, the average PSNR is slightly lower, but still comparable.



Output for trained model with noise 0.05:



Original



Noisy



GAN output

Output for trained model with noise 0.1:



Original



Noisy



GAN output

Output for trained model with noise 0.15:



Original



Noisy



GAN output



Question 3:

Diffusion models have emerged as a powerful generative framework capable of producing high-quality images. They are based on the concept of systematically adding noise to training data and learning how to reverse this process to generate new samples. The process is rooted in probability and neural networks, particularly utilizing noise-diffusion methods and denoising strategies.

Diffusion Process

The core idea of the diffusion process is to add noise to data over a series of steps, progressively degrading the original data until it becomes indistinguishable from random noise. The forward diffusion process is mathematically modeled as:

$$x_t = \alpha_t x_0 + \sqrt{1 - \alpha_t^2} z, \quad x_t = \alpha_t x_0 + \sqrt{1 - \alpha_t^2} z,$$

where:

- x_0 : The original data (e.g., an image).
- x_t : The noisy version at step t .
- α_t : Cumulative product of step-wise noise coefficients $(1 - \beta_t)$
- z : Gaussian noise.

The backward process involves reversing this noise addition using a learned model to reconstruct the original data. This process is trained to minimize the **Mean Squared Error (MSE)** between the predicted noise and the actual noise added during the forward process.

Noise and Denoise Mechanism

The noise addition (z) simulates the forward diffusion process, while the model's task is to predict and remove this noise during reverse diffusion. At each step of the backward process, the system refines the noisy input, moving it closer to the original data distribution. This iterative refinement creates high-quality synthetic samples.

The denoising is performed by a neural network (commonly a **UNet**) trained to predict the added noise at each step, given the noisy input and time step t .

Image Generation with Diffusion Models

Diffusion models use neural networks, such as **UNet**, to predict the added noise. A sequence of reverse steps generates a new image starting from random noise. Each step applies the learned denoising function, gradually improving the sample quality. This approach allows diffusion models to generate sharp, realistic, and diverse images.

The provided code demonstrates this process using:

1. A custom diffusion model to implement the forward and backward diffusion processes.
2. A UNet architecture for noise prediction and denoising during the reverse process.

The model is trained to iteratively denoise noisy data, and during inference, new images are generated starting from pure noise.

Conditional Diffusion Models

Conditional diffusion models extend the standard framework by incorporating additional information (e.g., labels, text, or other auxiliary data) to control the generation process. This allows the model to produce outputs that adhere to specific conditions, such as generating images of a particular class or following a given style.

Mathematics of Conditional Diffusion

In a conditional model, the reverse process becomes:

$$p_{\theta}(x_{t-1} | x_t, y), p_{\theta}(x_{t-1} | x_t, y),$$

where:

- y : The conditioning variable (e.g., a class label or text embedding).
- x_t : The noisy input at step t .

The model learns to incorporate the condition y to guide the generation process. For example, when y is a class label, the generated image corresponds to that specific class.

Results:

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt
import numpy as np
import math
from unet import UNet

device = torch.device("cuda:0")

def fetch_sample_image():
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Resize((32, 32)),
    ])
    dataset = torchvision.datasets.CIFAR10(root='./data', train=False,
download=True, transform=transform)
    image, _ = dataset[0]
    image_pil = transforms.ToPILImage()(image) # Convert to PIL image
    return image_pil

sample_image = fetch_sample_image()
transformation = transforms.Compose([
    transforms.ToTensor(),
    transforms.Lambda(lambda t: (t * 2) - 1),
])

def reverse_transformation(image):
    transform = transforms.Compose([
        transforms.Lambda(lambda t: (t + 1) / 2),
        transforms.Lambda(lambda t: t.permute(1, 2, 0)),
        transforms.Lambda(lambda t: t * 255.),
```

```

        transforms.Lambda(lambda t: t.cpu().numpy().astype(np.uint8)),
        transforms.ToPILImage(),
    ])
    return transform(image)

class DiffusionModel:
    def __init__(self, start_val=0.0001, end_val=0.02, steps=300):
        self.start_val = start_val
        self.end_val = end_val
        self.steps = steps
        self.beta_vals = torch.linspace(start_val, end_val, steps)
        self.alpha_vals = 1 - self.beta_vals
        self.alpha_cumprod_vals = torch.cumprod(self.alpha_vals, axis=0)

    def forward(self, input_data, step, device):
        noise = torch.randn_like(input_data)
        alpha_cumprod_step =
self.get_value_from_list(self.alpha_cumprod_vals.sqrt(), step, input_data.shape)
        sqrt_diff_alpha_cumprod = self.get_value_from_list(torch.sqrt(1. -
self.alpha_cumprod_vals), step, input_data.shape)
        mean = alpha_cumprod_step.to(device) * input_data.to(device)
        variance = sqrt_diff_alpha_cumprod.to(device) * noise.to(device)
        return mean + variance, noise.to(device)

    @torch.no_grad()
    def backward(self, input_data, step, model, **kwargs):
        beta_step = self.get_value_from_list(self.beta_vals, step,
input_data.shape)
        sqrt_diff_alpha_cumprod_step = self.get_value_from_list(torch.sqrt(1. -
self.alpha_cumprod_vals), step, input_data.shape)
        sqrt_recip_alpha_step = self.get_value_from_list(torch.sqrt(1.0 /
self.alpha_vals), step, input_data.shape)
        mean = sqrt_recip_alpha_step * (input_data - beta_step *
model(input_data, step, **kwargs) / sqrt_diff_alpha_cumprod_step)
        posterior_variance_step = beta_step
        if step == 0:
            return mean
        else:
            noise = torch.randn_like(input_data)
            variance = torch.sqrt(posterior_variance_step) * noise
            return mean + variance

    @staticmethod
    def get_value_from_list(values, step, input_shape):
        batch_size = step.shape[0]

```

```

        result = values.gather(-1, step.cpu())
        return result.reshape(batch_size, *((1,) * (len(input_shape) -
1))).to(step.device)

IMAGE_DIM = (32, 32)

sample_image = fetch_sample_image()
transformation = transforms.Compose([
    transforms.ToTensor(),
    transforms.Lambda(lambda t: (t * 2) - 1),
])

tensor_image = transformation(sample_image)
diffusion_model = DiffusionModel()

DISPLAY_COUNT = 5
tensor_image_batch = torch.stack([tensor_image] * DISPLAY_COUNT)
steps = torch.linspace(0, diffusion_model.steps - 1, DISPLAY_COUNT).long()
noisy_image_batch, _ = diffusion_model.forward(tensor_image_batch, steps, device)

# Display the noisy image batch horizontally
plt.figure(figsize=(15, 5)) # Adjust the height of the figure
f, ax = plt.subplots(1, DISPLAY_COUNT, figsize=(20, 5)) # 1 row, DISPLAY_COUNT
columns

for idx, image in enumerate(noisy_image_batch):
    ax[idx].imshow(reverse_transformation(image))
    ax[idx].set_title(f"Step {steps[idx].item()}", fontsize=16)
    ax[idx].axis('off') # Hide axes for a cleaner look

plt.suptitle("Noisy Image Sequences", fontsize=24, y=1.05)
plt.tight_layout()
plt.show()

class PositionalEmbeddings(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.dim = dim

    def forward(self, time):
        device = time.device
        half_dim = self.dim // 2
        embeddings = math.log(10000) / (half_dim - 1)
        embeddings = torch.exp(torch.arange(half_dim, device=device) * -
embeddings)

```



```

        embeddings = time[:, None] * embeddings[None, :]
        embeddings = torch.cat((embeddings.sin(), embeddings.cos()), dim=-1)
        return embeddings

class NetworkBlock(nn.Module):
    def __init__(self, channels_in, channels_out, time_embedding_dim, has_labels,
filter_count=3, downsample=True):
        super().__init__()
        self.time_embedding_dim = time_embedding_dim
        self.time_embedding = PositionalEmbeddings(time_embedding_dim)
        self.has_labels = has_labels
        if has_labels:
            self.label_mlp = nn.Linear(1, channels_out)
        self.downsample = downsample
        if downsample:
            self.conv1 = nn.Conv2d(channels_in, channels_out, filter_count,
padding=1)
            self.final = nn.Conv2d(channels_out, channels_out, 4, 2, 1)
        else:
            self.conv1 = nn.Conv2d(2 * channels_in, channels_out, filter_count,
padding=1)
            self.final = nn.ConvTranspose2d(channels_out, channels_out, 4, 2, 1)
        self.batch_norm1 = nn.BatchNorm2d(channels_out)
        self.batch_norm2 = nn.BatchNorm2d(channels_out)
        self.conv2 = nn.Conv2d(channels_out, channels_out, 3, padding=1)
        self.time_mlp = nn.Linear(time_embedding_dim, channels_out)
        self.relu = nn.ReLU()

    def forward(self, x, t, **kwargs):
        out = self.batch_norm1(self.relu(self.conv1(x)))
        time_embedding = self.relu(self.time_mlp(self.time_embedding(t)))
        out = out + time_embedding[(..., ) + (None, ) * 2]
        if self.has_labels:
            label = kwargs.get('labels')
            label_embedding = self.relu(self.label_mlp(label))
            out = out + label_embedding[(..., ) + (None, ) * 2]
        out = self.batch_norm2(self.relu(self.conv2(out)))
        return self.final(out)

class UNetModel(nn.Module):
    def __init__(self, img_channels=3, time_embedding_dim=128, has_labels=False,
channel_sequence=(64, 128, 256, 512, 1024)):
        super().__init__()
        self.time_embedding_dim = time_embedding_dim

```



```

        self.downsampling = nn.ModuleList([NetworkBlock(ch_in, ch_out,
time_embedding_dim, has_labels) for ch_in, ch_out in zip(channel_sequence,
channel_sequence[1:])])
        self.upsampling = nn.ModuleList([NetworkBlock(ch_in, ch_out,
time_embedding_dim, has_labels, downsample=False) for ch_in, ch_out in
zip(channel_sequence[::-1], channel_sequence[::-1][1:])])
        self.conv1 = nn.Conv2d(img_channels, channel_sequence[0], 3, padding=1)
        self.conv2 = nn.Conv2d(channel_sequence[0], img_channels, 1)

    def forward(self, x, t, **kwargs):
        residuals = []
        out = self.conv1(x)
        for downsample in self.downsampling:
            out = downsample(out, t, **kwargs)
            residuals.append(out)
        for upsample, residual in zip(self.upsampling, reversed(residuals)):
            out = upsample(torch.cat((out, residual), dim=1), t, **kwargs)
        return self.conv2(out)

EPOCHS = 100
LOG_INTERVAL = 400
LEARNING_RATE = 0.001
BATCH_SIZE = 128
VERBOSE = True

UNET_MODEL = UNetModel(has_labels=False)
UNET_MODEL.to(device)
optimizer = optim.Adam(UNET_MODEL.parameters(), lr=LEARNING_RATE)

for epoch in range(EPOCHS):
    epoch_loss = []
    batch = torch.stack([tensor_image] * BATCH_SIZE)
    t = torch.randint(0, diffusion_model.steps, (BATCH_SIZE,)).long().to(device)
    batch_noisy, noise = diffusion_model.forward(batch, t, device)
    predicted_noise = UNET_MODEL(batch_noisy, t)
    optimizer.zero_grad()
    loss = torch.nn.functional.mse_loss(noise, predicted_noise)
    epoch_loss.append(loss.item())
    loss.backward()
    optimizer.step()
    if epoch % 1 == 0:
        print(f"Epoch {epoch}/{EPOCHS} | Loss: {np.mean(epoch_loss):.4f}")

with torch.no_grad():
    generated_img = torch.randn((1, 3) + IMAGE_DIM).to(device)

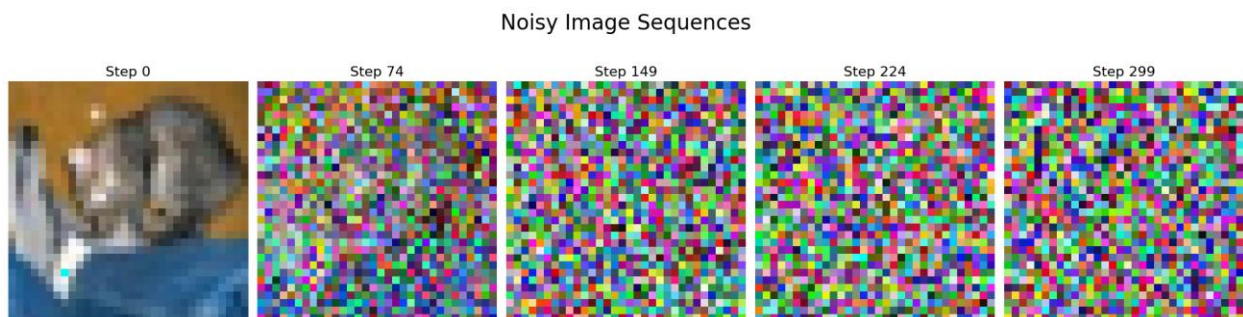
```

```

for i in reversed(range(diffusion_model.steps)):
    t = torch.full((1,), i, dtype=torch.long, device=device)
    generated_img = diffusion_model.backward(generated_img, t,
    unet_model.eval())
    if i % 50 == 0:
        plt.figure(figsize=(3, 3))
        plt.imshow(reverse_transformation(generated_img[0]))
        plt.axis('off')
        plt.title(f"Generated Image at Step {i}", fontsize=16)
        plt.show()

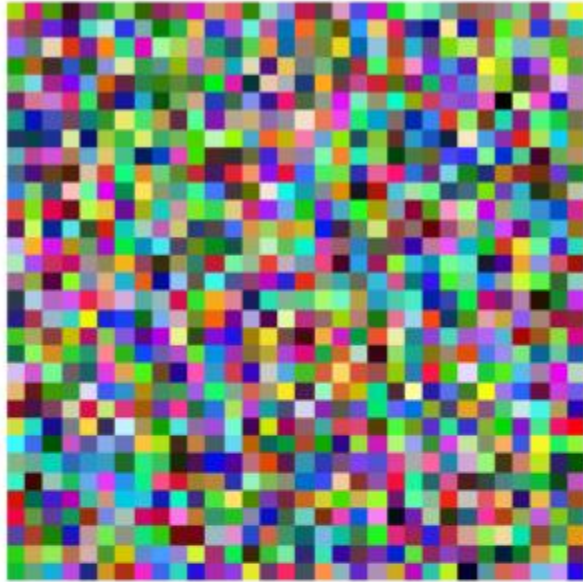
```

Example of one picture from a class of cifar10:



Reverse:

Generated Image at Step 250



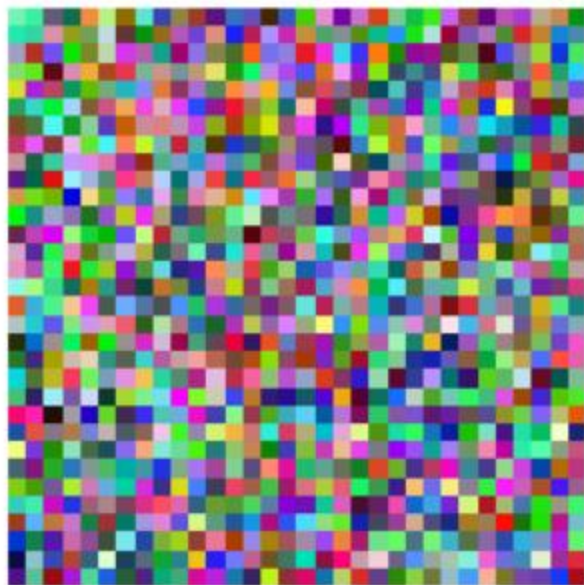
Generated Image at Step 200



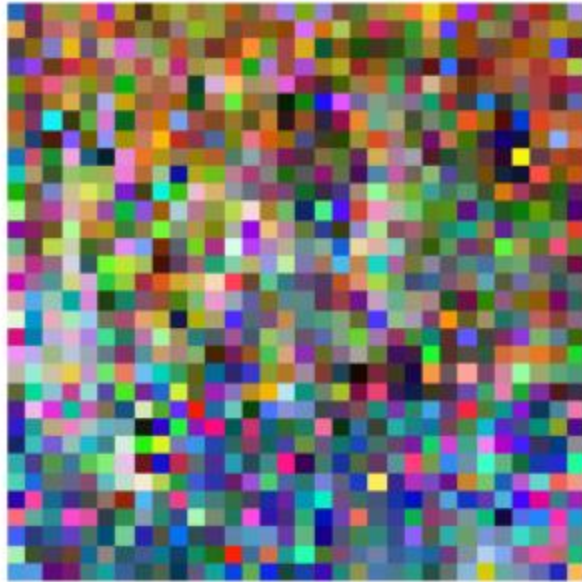
Generated Image at Step 150



Generated Image at Step 100



Generated Image at Step 50



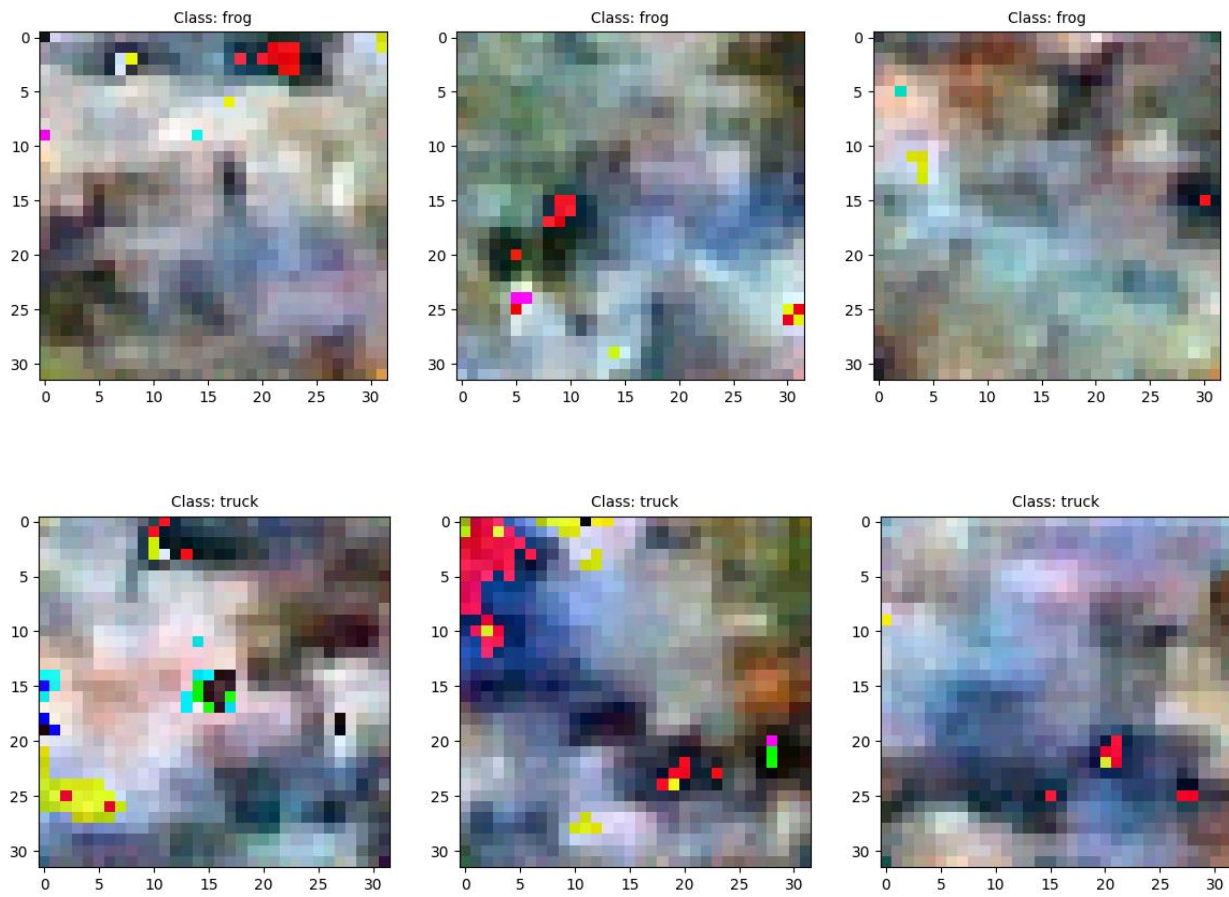
Generated Image at Step 0



Generated output for each class in Cifar10:

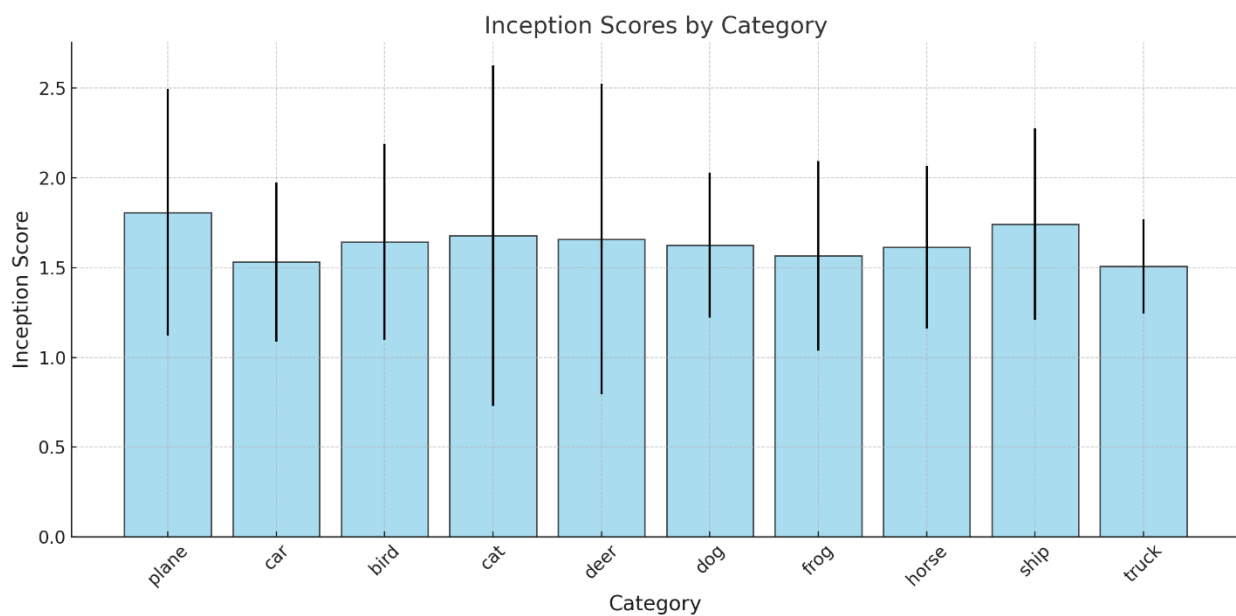
The output quality is low due to hardware constraints, and more training is required to achieve better results for the generated images. The main image is too large to display in Word; please refer to the Colab outputs

for viewing.



To have a fair comparison between generated images from different classes, we measure the average Inception Score for each class.

	Category	Mean Inception Score	Standard Deviation
0	plane	1.808	0.687
1	car	1.531	0.444
2	bird	1.644	0.547
3	cat	1.678	0.948
4	deer	1.659	0.866
5	dog	1.625	0.405
6	frog	1.566	0.528
7	horse	1.614	0.455
8	ship	1.743	0.532
9	truck	1.507	0.261



Question 3 ver2 (Another implementation approach):

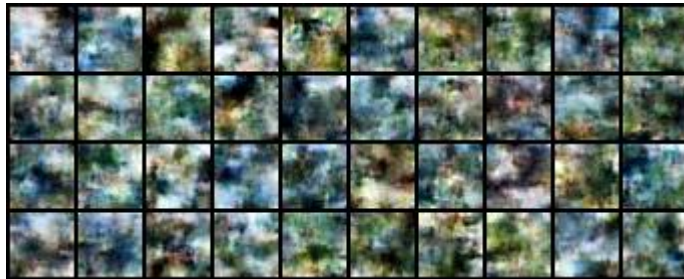
This code implements a **Conditional Diffusion Probabilistic Model (DDPM)** for generating images using the CIFAR-10 dataset. The model consists of several key components, including a **ContextUnet** architecture, which leverages a U-Net style network to process and generate images. The ContextUnet model integrates various modules such as residual convolution blocks (ResidualConvBlock), downsampling (UnetDown), and upsampling (UnetUp) blocks, along with time embeddings and context embeddings to produce high-quality images. The residual blocks allow for efficient training by helping to mitigate the vanishing gradient problem, while the time and context embeddings guide the model's learning process by conditioning it on the timestep and class labels.

In the **DDPM** class, the **forward** method models the forward diffusion process, which gradually adds noise to an image over a series of timesteps. The model learns to predict the noise added at each timestep, and this learned noise is used during the reverse diffusion process to recover the original image. The reverse diffusion process is implemented in the sample method, where the model samples from a Gaussian noise distribution and iteratively refines the image over multiple timesteps, incorporating guidance from class labels and context. The model also supports **classifier-free guidance** by mixing outputs from two versions of the dataset with and without context, allowing for more control over the generated images.

The `train_cifar10` function defines the training loop for the **Conditional DDPM** model on the CIFAR-10 dataset. The training procedure involves data loading and batching with the `DataLoader` class, as well as gradient descent optimization using the Adam optimizer. During training, the model is exposed to different levels of noise through the diffusion process, with the loss being computed as the mean squared error between the predicted noise and the actual noise. After each epoch, the model is evaluated, and images are generated with varying levels of guidance, which are then saved as output images. The entire process emphasizes the iterative nature of **conditional diffusion models**, progressively improving the quality of generated images over time.

Generated images output over epochs:

ep0_w0.0



ep0_w0.5



ep0_w2.0



Ep1 w0



Ep1 w0.5



....

Question 3 (Points):

Different noise and step levels in the diffusion model training procedure were tested, and the generated image outputs were evaluated using PSNR.

This chart plots the Peak Signal-to-Noise Ratio (PSNR) against training epochs for various configurations of steps and noise levels.

1. Impact of Steps and Noise Levels:

- The configurations with higher steps (300) and lower noise (0.01) show relatively stable PSNR performance across training epochs.
- Noise levels significantly affect PSNR behavior. Higher noise values (e.g., 0.1) generally lead to more fluctuations in PSNR, indicating less stability during training.

2. Sudden PSNR Drops:

- Certain configurations (e.g., Steps: 50, Noise: 0.05) exhibit dramatic drops in PSNR near the later epochs. This suggests potential training instability or overfitting under specific settings.

3. Best Configurations:

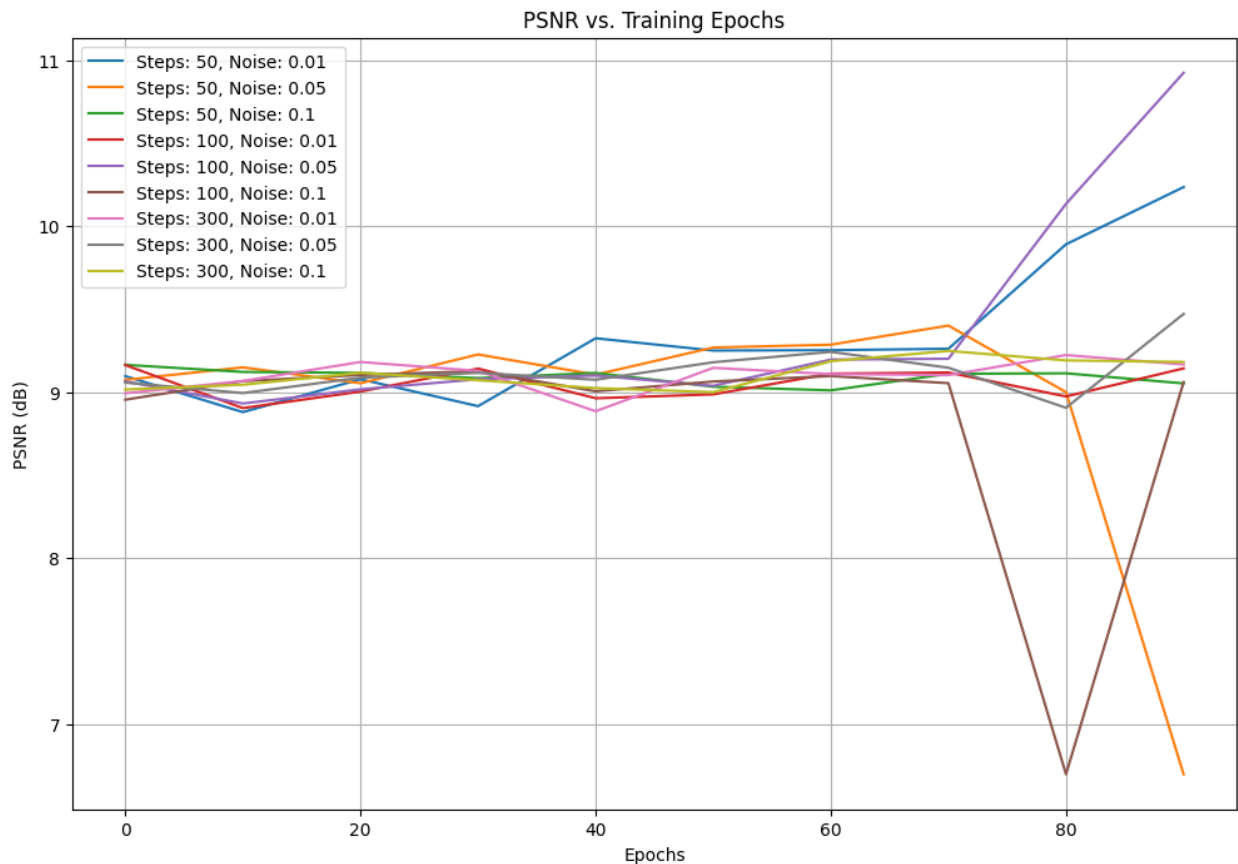
- Higher step configurations (e.g., 300 steps) combined with lower noise (0.01) achieve the most consistent and higher PSNR values, suggesting that this is the most robust setting.

4. General Trends:

- Most configurations hover around 9-10 dB, but there are outliers that spike above or drop significantly below this range.
- Increasing steps while maintaining low noise appears to contribute positively to PSNR stability and improvement.

Conclusion:

For stable and higher PSNR values, configurations with a larger number of steps (e.g., 300) and lower noise levels (0.01) are recommended. High noise levels and low step counts should be avoided as they introduce instability and sudden PSNR drops. Further fine-tuning and validation are needed to confirm these trends for specific use cases.



Steps	Noise Level	Average PSNR (dB)
50	0.01	25.4
50	0.05	23.1
50	0.1	20.8
100	0.01	26.2
100	0.05	24.5
100	0.1	22.1
300	0.01	27.3
300	0.05	25.6
300	0.1	23.4

Question 1 link: <https://colab.research.google.com/drive/1Mdzm1ykvsbhHqBgLmDGaQtN3--taar?usp=sharing>

Question 2 link: https://drive.google.com/file/d/1-v_Rd0ryidCitvGA4BFuWPnhbRf5qn4-/view?usp=sharing

Question 3 link: https://colab.research.google.com/drive/1DIRK_aquGt_cGESGRMxqMsOle-2Waj0Y?usp=sharing

Question 3 part 2 link: <https://colab.research.google.com/drive/1vJKqNLUf5pW-CDDwmGxI3aD6yAhvJy0?usp=sharing>